FREVIA – Report 3

Nguyen Lu Nhat Huy – Sprints 1–5

Updated 08 November 2026

Contents

1. Use Case Specifications 4
   1.1 Owned Use Cases 4
      1.1.1 Sprint 1–5 4
2. Functional Requirements 70
   2.1 Freelancer Profile Management 70
      2.1.1 Function: View Freelancer Profile (UC-07.01) 70
      2.1.2 Function: Update Freelancer Profile (UC-07.02) 73
   2.2 Freelancer Skills Management 76
      2.2.1 Function: View Freelancer Skill List (UC-08.01) 76
      2.2.2 Function: Add Freelancer Skill (UC-08.02) 78
      2.2.3 Function: Delete Freelancer Skill (UC-08.03) 81
   2.3 Portfolio Management 83
      2.3.1 Function: View Portfolio List (UC-09.01) 83
      2.3.2 Function: Add Portfolio (UC-09.02) 86
      2.3.3 Function: View Portfolio Detail (UC-09.03) 89
      2.3.4 Function: Update Portfolio (UC-09.04) 91
      2.3.5 Function: Delete Portfolio (UC-09.05) 94
   2.4 Identity Verification 96
      2.4.1 Function: Upload Identity Documents (UC-18.01) 96
      2.4.2 Function: View Verification Status (UC-18.02) 100
   2.5 Client Profile Management 103
      2.5.1 Function: View Client Profile (UC-19.01) 103
      2.5.2 Function: Update Company Information (UC-19.02) 105
   2.6 Social Links Management 109
      2.6.1 Function: View Social Link List (UC-20.01) 109
      2.6.2 Function: Add Social Link (UC-20.02) 111
      2.6.3 Function: Delete Social Link (UC-20.03) 114
   2.7 Favorite Freelancers 117
      2.7.1 Function: View Favorite Freelancer List (UC-21.01) 117
      2.7.2 Function: Add Freelancer to Favorites (UC-21.02) 119
      2.7.3 Function: Remove Freelancer from Favorites (UC-21.03) 122

2.8 Following Management . . . 125
2.8.1 Function: View Following List (UC-29.01) . . . 125
2.8.2 Function: Follow Freelancer (UC-29.02) . . . 127
2.8.3 Function: Unfollow Freelancer (UC-29.03) . . . 130
2.9 Notification Management . . . 132
2.9.1 Function: View Notification List (UC-30.01) . . . 132
2.9.2 Function: Mark Notification as Read (UC-30.02) . . . 135
2.9.3 Function: Mark All Notifications as Read (UC-30.03) . . . 137
2.9.4 Function: Delete Notification (UC-30.04) . . . 140
2.10 Role Switching and Enrollment . . . 142
2.10.1 Function: Switch Role (UC-31.01) . . . 142
2.10.2 Function: Join New Role (UC-31.02) . . . 145
2.11 Contract Review Management . . . 148
2.11.1 Function: View Review List (UC-39.01) . . . 148
2.11.2 Function: View Review Detail (UC-39.02) . . . 150
2.11.3 Function: Write Review (UC-39.03) . . . 153
2.11.4 Function: Update Review (UC-39.04) . . . 156
2.11.5 Function: Delete Review (UC-39.05) . . . 159
2.12 Review Response Management . . . 162
2.12.1 Function: Respond to Review (UC-40.01) . . . 162
2.12.2 Function: Update Review Response (UC-40.02) . . . 164
2.12.3 Function: Delete Review Response (UC-40.03) . . . 167
2.13 General Account Profile . . . 169
2.13.1 Function: View Profile (UC-41.01) . . . 169
2.13.2 Function: Update Profile (UC-41.02) . . . 172
2.13.3 Function: Change Password (UC-41.03) . . . 174
2.13.4 Function: Upload Avatar (UC-41.04) . . . 177
2.14 Profile Revision Review and Expert Profile . . . 180
2.14.1 Function: View Pending Profile Update Request List (UC-47.01) . . . 180
2.14.2 Function: View Profile Update Request Detail (UC-47.02) . . . 182
2.14.3 Function: Approve Profile Update Request (UC-47.03) . . . 186
2.14.4 Function: Reject Profile Update Request (UC-47.04) . . . 189
2.14.5 Function: View Profile Update Request History (UC-47.05) . . . 192
2.14.6 Function: Update Profile Expert (UC-47.06) . . . 195
2.15 Freelancer CV Management . . . 198
2.15.1 Function: Upload CV (UC-07.03) . . . 198
2.15.2 Function: View CV (UC-07.04) . . . 201
2.15.3 Function: Replace CV (UC-07.05) . . . 204

2.15.4 Function: Delete CV (UC-07.06) 207
2.16 Nhuy Requirement Traceability and Acceptance Scope 209
2.16.1 Acceptance Gaps Requiring Verification 210
2.16.2 Schema and Identifier Conventions 211

1. Use Case Specifications

1.1 Owned Use Cases

1.1.1 Sprint 1–5

1.1.1.1 Freelancer Profile Management

UC-07.01 View Freelancer Profile

UC ID and Name	UC-07.01 View Freelancer Profile
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Guest, Client, Freelancer
Trigger:	The actor selects View Freelancer Profile from /freelancer/profile; /profiles/[id].
Description:	Retrieve approved Profile and FreelancerProfile fields in one response.
Preconditions:	The requested active profile or item exists. The relevant database and dependent services are available.
Post-conditions:	Authorized data is displayed without modifying the underlying records.
Normal Flow:	1. The actor opens /freelancer/profile; /profiles/[id] and selects View Freelancer Profile. 2. The application submits the selected identifier or list filters. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Retrieve approved Profile and FreelancerProfile fields in one response. 5. The screen renders the response, or an explicit empty state for a valid empty collection.
Alternative Flows:	No matching records: display an empty state. A failed request offers Retry.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **A1:** A valid empty collection displays an empty state; it must not be treated as a missing account. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.

Business Rules:	**BR-NH-07-01:** Use Profile.id for profile routes; User.id identifies the owner and must not be substituted for the profile ID. **BR-NH-07-02:** Only the owner changes professional data. Public output excludes credentials and private CV locations. **BR-NH-07-03:** Omitted optional fields are preserved. An explicit null or empty list clears the corresponding value. **BR-NH-07-04:** Freelancer and Client updates below 20 percent profile strength require a pending revision; other updates apply directly. Pending changes are not shown as approved public data.
Other Information:	`GET /profiles/:id`. Persistent entities: Profile, FreelancerProfile, ProfileRevision.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-07.02 Update Freelancer Profile

UC ID and Name	UC-07.02 Update Freelancer Profile
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Freelancer
Trigger:	The actor selects Update Freelancer Profile from /freelancer/profile; /profiles/[id].
Description:	Persist basic and professional fields together or create a pending revision under the review policy.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Persist basic and professional fields together or create a pending revision under the review policy.
Normal Flow:	1. The actor opens /freelancer/profile; /profiles/[id] and selects Update Freelancer Profile. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Persist basic and professional fields together or create a pending revision under the review policy. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-07-01:** Use Profile.id for profile routes; User.id identifies the owner and must not be substituted for the profile ID. **BR-NH-07-02:** Only the owner changes professional data. Public output excludes credentials and private CV locations. **BR-NH-07-03:** Omitted optional fields are preserved. An explicit null or empty list clears the corresponding value. **BR-NH-07-04:** Freelancer and Client updates below 20 percent profile strength require a pending revision; other updates apply directly. Pending changes are not shown as approved public data.
Other Information:	`PUT /profiles/:id`. Persistent entities: Profile, FreelancerProfile, ProfileRevision.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

1.1.1.2 Freelancer Skills Management

UC-08.01 View Freelancer Skill List

UC ID and Name	UC-08.01 View Freelancer Skill List
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Guest, Client, Freelancer
Trigger:	The actor selects View Freelancer Skill List from /profiles/[id]; /freelancer/profile.
Description:	Join FreelancerSkill with active Skill catalog entries and show proficiency levels.
Preconditions:	The requested active profile or item exists. The relevant database and dependent services are available.

Post-conditions:	Authorized data is displayed without modifying the underlying records.
Normal Flow:	1. The actor opens /profiles/[id]; /freelancer/profile and selects View Freelancer Skill List. 2. The application submits the selected identifier or list filters. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Join FreelancerSkill with active Skill catalog entries and show proficiency levels. 5. The screen renders the response, or an explicit empty state for a valid empty collection.
Alternative Flows:	No matching records: display an empty state. A failed request offers Retry.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **A1:** A valid empty collection displays an empty state; it must not be treated as a missing account. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-08-01:** A freelancer can add and delete mappings only on their own profile. **BR-NH-08-02:** The pair (freelancerProfileId, skillId) must be unique, including concurrent requests. **BR-NH-08-03:** Deleting a skill permanently removes FreelancerSkill but retains the shared Skill definition. **BR-NH-08-04:** Show an empty state when there are no active skills.
Other Information:	`GET /profiles/:id/skills`. Persistent entities: Profile, FreelancerProfile, FreelancerSkill, Skill.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-08.02 Add Freelancer Skill

UC ID and Name	UC-08.02 Add Freelancer Skill
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Freelancer
Trigger:	The actor selects Add Freelancer Skill from /profiles/[id]; /freelancer/profile.
Description:	Resolve or create the canonical catalog skill and insert one owned FreelancerSkill mapping.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Resolve or create the canonical catalog skill and insert one owned FreelancerSkill mapping.
Normal Flow:	1. The actor opens /profiles/[id]; /freelancer/profile and selects Add Freelancer Skill. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Resolve or create the canonical catalog skill and insert one owned FreelancerSkill mapping. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-08-01:** A freelancer can add and delete mappings only on their own profile. **BR-NH-08-02:** The pair (freelancerProfileId, skillId) must be unique, including concurrent requests. **BR-NH-08-03:** Deleting a skill permanently removes FreelancerSkill but retains the shared Skill definition. **BR-NH-08-04:** Show an empty state when there are no active skills.

Other Information:	`POST /profiles/:id/skills`. Persistent entities: Profile, FreelancerProfile, FreelancerSkill, Skill.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-08.03 Delete Freelancer Skill

UC ID and Name	UC-08.03 Delete Freelancer Skill
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Freelancer
Trigger:	The actor selects Delete Freelancer Skill from /profiles/[id]; /freelancer/profile.
Description:	Hard-delete the owned mapping while retaining its Skill catalog entry.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Hard-delete the owned mapping while retaining its Skill catalog entry.
Normal Flow:	1. The actor opens /profiles/[id]; /freelancer/profile and selects Delete Freelancer Skill. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Hard-delete the owned mapping while retaining its Skill catalog entry. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-08-01:** A freelancer can add and delete mappings only on their own profile. **BR-NH-08-02:** The pair (freelancerProfileId, skillId) must be unique, including concurrent requests. **BR-NH-08-03:** Deleting a skill permanently removes FreelancerSkill but retains the shared Skill definition. **BR-NH-08-04:** Show an empty state when there are no active skills.
Other Information:	`DELETE /profiles/skills/:id`. Persistent entities: Profile, FreelancerProfile, FreelancerSkill, Skill.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

1.1.1.3 Portfolio Management

UC-09.01 View Portfolio List

UC ID and Name	UC-09.01 View Portfolio List
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Guest, Client, Freelancer
Trigger:	The actor selects View Portfolio List from /profiles/[id]; /freelancer/profile.
Description:	List only non-deleted PortfolioItem records of the selected freelancer.
Preconditions:	The requested active profile or item exists. The relevant database and dependent services are available.
Post-conditions:	Authorized data is displayed without modifying the underlying records.
Normal Flow:	1. The actor opens /profiles/[id]; /freelancer/profile and selects View Portfolio List. 2. The application submits the selected identifier or list filters. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. List only non-deleted PortfolioItem records of the selected freelancer. 5. The screen renders the response, or an explicit empty state for a valid empty collection.

Alternative Flows:	No matching records: display an empty state. A failed request offers Retry.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **A1:** A valid empty collection displays an empty state; it must not be treated as a missing account. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-09-01:** Only the owning freelancer creates, updates or deletes a portfolio item. **BR-NH-09-02:** Only items whose deletedAt is null are listed, opened or edited. **BR-NH-09-03:** Deletion sets deletedAt; it does not hard-delete the item. **BR-NH-09-04:** The agreed database scope is title, description, technologies and project URL; media gallery, uploads and project role are excluded.
Other Information:	`GET /profiles/:id/portfolios`. Persistent entities: Profile, FreelancerProfile, PortfolioItem.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-09.02 Add Portfolio

UC ID and Name	UC-09.02 Add Portfolio
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Freelancer
Trigger:	The actor selects Add Portfolio from /profiles/[id]; /freelancer/profile.
Description:	Create an owned portfolio item with title, description, technologies and project link.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Create an owned portfolio item with title, description, technologies and project link.

Normal Flow:	1. The actor opens /profiles/[id]; /freelancer/profile and selects Add Portfolio. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Create an owned portfolio item with title, description, technologies and project link. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-09-01:** Only the owning freelancer creates, updates or deletes a portfolio item. **BR-NH-09-02:** Only items whose deletedAt is null are listed, opened or edited. **BR-NH-09-03:** Deletion sets deletedAt; it does not hard-delete the item. **BR-NH-09-04:** The agreed database scope is title, description, technologies and project URL; media gallery, uploads and project role are excluded.
Other Information:	`POST /profiles/:id/portfolios`. Persistent entities: Profile, FreelancerProfile, PortfolioItem.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-09.03 View Portfolio Detail

UC ID and Name	UC-09.03 View Portfolio Detail
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Guest, Client, Freelancer
Trigger:	The actor selects View Portfolio Detail from /profiles/[id]; /freelancer/profile.
Description:	Display a single non-deleted portfolio item and its external project link.
Preconditions:	The requested active profile or item exists. The relevant database and dependent services are available.
Post-conditions:	Authorized data is displayed without modifying the underlying records.
Normal Flow:	1. The actor opens /profiles/[id]; /freelancer/profile and selects View Portfolio Detail. 2. The application submits the selected identifier or list filters. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Display a single non-deleted portfolio item and its external project link. 5. The screen renders the response, or an explicit empty state for a valid empty collection.
Alternative Flows:	No matching records: display an empty state. A failed request offers Retry.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **A1:** A valid empty collection displays an empty state; it must not be treated as a missing account. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-09-01:** Only the owning freelancer creates, updates or deletes a portfolio item. **BR-NH-09-02:** Only items whose deletedAt is null are listed, opened or edited. **BR-NH-09-03:** Deletion sets deletedAt; it does not hard-delete the item. **BR-NH-09-04:** The agreed database scope is title, description, technologies and project URL; media gallery, uploads and project role are excluded.

Other Information:	`GET /profiles/portfolios/:id`. Persistent entities: Profile, FreelancerProfile, PortfolioItem.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-09.04 Update Portfolio

UC ID and Name	UC-09.04 Update Portfolio
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Freelancer
Trigger:	The actor selects Update Portfolio from /profiles/[id]; /freelancer/profile.
Description:	Save validated edits on an owned, non-deleted portfolio item.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Save validated edits on an owned, non-deleted portfolio item.
Normal Flow:	1. The actor opens /profiles/[id]; /freelancer/profile and selects Update Portfolio. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Save validated edits on an owned, non-deleted portfolio item. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.

Business Rules:	**BR-NH-09-01:** Only the owning freelancer creates, updates or deletes a portfolio item. **BR-NH-09-02:** Only items whose deletedAt is null are listed, opened or edited. **BR-NH-09-03:** Deletion sets deletedAt; it does not hard-delete the item. **BR-NH-09-04:** The agreed database scope is title, description, technologies and project URL; media gallery, uploads and project role are excluded.
Other Information:	`PUT /profiles/portfolios/:id`. Persistent entities: Profile, FreelancerProfile, PortfolioItem.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-09.05 Delete Portfolio

UC ID and Name	UC-09.05 Delete Portfolio
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Freelancer
Trigger:	The actor selects Delete Portfolio from /profiles/[id]; /freelancer/profile.
Description:	Set deletedAt and remove the item from active list and detail results.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Set deletedAt and remove the item from active list and detail results.
Normal Flow:	1. The actor opens /profiles/[id]; /freelancer/profile and selects Delete Portfolio. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Set deletedAt and remove the item from active list and detail results. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-09-01:** Only the owning freelancer creates, updates or deletes a portfolio item. **BR-NH-09-02:** Only items whose deletedAt is null are listed, opened or edited. **BR-NH-09-03:** Deletion sets deletedAt; it does not hard-delete the item. **BR-NH-09-04:** The agreed database scope is title, description, technologies and project URL; media gallery, uploads and project role are excluded.
Other Information:	`DELETE /profiles/portfolios/:id`. Persistent entities: Profile, FreelancerProfile, PortfolioItem.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

1.1.1.4 Identity Verification

UC-18.01 Upload Identity Documents

UC ID and Name	UC-18.01 Upload Identity Documents
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Freelancer
Trigger:	The actor selects Upload Identity Documents from /account-profile (Identity Verification tab).
Description:	Validate the active-request constraint, securely store the submitted evidence, and record the verification outcome for review.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.

Post-conditions:	Validate the active-request constraint, securely store the submitted evidence, and record the verification outcome for review.
Normal Flow:	1. The actor opens /account-profile (Identity Verification tab) and selects Upload Identity Documents. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Validate the active-request constraint, securely store the submitted evidence, and record the verification outcome for review. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process. **E5:** An active PENDING/APPROVED request blocks resubmission. An unavailable AI service is reported without creating a successful verification record.
Priority:	High
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-18-01:** Only an authenticated freelancer submits or reads their own verification status. **BR-NH-18-02:** At most one active PENDING or APPROVED request is allowed per freelancer. **BR-NH-18-03:** Documents are confidential; storage delivery and file endpoints must enforce authorization. **BR-NH-18-04:** AI success creates PENDING for administrator review; AI rejection records REJECTED with a reason. AI transport failure must not create a successful request. **BR-NH-18-05:** Rejected users may resubmit after correcting the stated problem. NOT_SUBMITTED is displayed when API status is null.
Other Information:	`POST /identity-verifications/ai-check; POST /identity-verifications/documents`. Persistent entities: User, IdVerificationDocument, FreelancerProfile.

Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-18.02 View Verification Status

UC ID and Name	UC-18.02 View Verification Status
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Freelancer
Trigger:	The actor selects View Verification Status from /account-profile (Identity Verification tab).
Description:	Return the latest verification state and submission history with rejection notes where present.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Authorized data is displayed without modifying the underlying records.
Normal Flow:	1. The actor opens /account-profile (Identity Verification tab) and selects View Verification Status. 2. The application submits the selected identifier or list filters. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Return the latest verification state and submission history with rejection notes where present. 5. The screen renders the response, or an explicit empty state for a valid empty collection.
Alternative Flows:	No matching records: display an empty state. A failed request offers Retry.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **A1:** A valid empty collection displays an empty state; it must not be treated as a missing account. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process. **E5:** An active PENDING/APPROVED request blocks resubmission. An unavailable AI service is reported without creating a successful verification record.
Priority:	High

Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-18-01:** Only an authenticated freelancer submits or reads their own verification status. **BR-NH-18-02:** At most one active PENDING or APPROVED request is allowed per freelancer. **BR-NH-18-03:** Documents are confidential; storage delivery and file endpoints must enforce authorization. **BR-NH-18-04:** AI success creates PENDING for administrator review; AI rejection records REJECTED with a reason. AI transport failure must not create a successful request. **BR-NH-18-05:** Rejected users may resubmit after correcting the stated problem. NOT_SUBMITTED is displayed when API status is null.
Other Information:	`GET /identity-verifications/status`. Persistent entities: User, IdVerificationDocument, FreelancerProfile.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

1.1.1.5 Client Profile Management

UC-19.01 View Client Profile

UC ID and Name	UC-19.01 View Client Profile
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Guest, Client, Freelancer
Trigger:	The actor selects View Client Profile from /clients/[userId]; /client/profile.
Description:	Combine approved public Profile, ClientProfile and SocialLink fields without private user information.
Preconditions:	The requested active profile or item exists. The relevant database and dependent services are available.
Post-conditions:	Authorized data is displayed without modifying the underlying records.

Normal Flow:	1. The actor opens /clients/[userId]; /client/profile and selects View Client Profile. 2. The application submits the selected identifier or list filters. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Combine approved public Profile, ClientProfile and SocialLink fields without private user information. 5. The screen renders the response, or an explicit empty state for a valid empty collection.
Alternative Flows:	No matching records: display an empty state. A failed request offers Retry.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **A1:** A valid empty collection displays an empty state; it must not be treated as a missing account. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-19-01:** Public client output contains approved company and profile fields, never password hashes, payment tokens or private identity documents. **BR-NH-19-02:** Only the authenticated Client updates their own company; the server derives the user ID from authentication. **BR-NH-19-03:** Apply the profile revision policy when strength is below 20 percent. **BR-NH-19-04:** Industry and location are outside the current ClientProfile schema. Hiring statistics are not part of the current response contract.
Other Information:	`GET /clients/:userId`. Persistent entities: User, Profile, ClientProfile, SocialLink, ProfileRevision.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-19.02 Update Company Information

UC ID and Name	UC-19.02 Update Company Information
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client
Trigger:	The actor selects Update Company Information from /clients/[userId]; /client/profile.
Description:	Save or submit the authenticated Client company information according to the revision policy.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Save or submit the authenticated Client company information according to the revision policy.
Normal Flow:	1. The actor opens /clients/[userId]; /client/profile and selects Update Company Information. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Save or submit the authenticated Client company information according to the revision policy. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.

Business Rules:	**BR-NH-19-01:** Public client output contains approved company and profile fields, never password hashes, payment tokens or private identity documents. **BR-NH-19-02:** Only the authenticated Client updates their own company; the server derives the user ID from authentication. **BR-NH-19-03:** Apply the profile revision policy when strength is below 20 percent. **BR-NH-19-04:** Industry and location are outside the current ClientProfile schema. Hiring statistics are not part of the current response contract.
Other Information:	`PUT /clients/me/profile`. Persistent entities: User, Profile, ClientProfile, SocialLink, ProfileRevision.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

1.1.1.6 Social Links Management

UC-20.01 View Social Link List

UC ID and Name	UC-20.01 View Social Link List
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer
Trigger:	The actor selects View Social Link List from /account-profile (Social Links); /client/profile; public profile.
Description:	Retrieve links for the current account settings or the public client response; public freelancer rendering is a tracked gap.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Authorized data is displayed without modifying the underlying records.
Normal Flow:	1. The actor opens /account-profile (Social Links); /client/profile; public profile and selects View Social Link List. 2. The application submits the selected identifier or list filters. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Retrieve links for the current account settings or the public client response; public freelancer rendering is a tracked gap. 5. The screen renders the response, or an explicit empty state for a valid empty collection.

Alternative Flows:	No matching records: display an empty state. A failed request offers Retry.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **A1:** A valid empty collection displays an empty state; it must not be treated as a missing account. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-20-01:** At most one link per platform per profile is allowed, including concurrent submissions. **BR-NH-20-02:** Only the owning user can add or hard-delete a social link. **BR-NH-20-03:** Open approved external links in a new tab with rel="noopener noreferrer". **BR-NH-20-04:** Profile.id links the SocialLink table; the authenticated User.id resolves that profile.
Other Information:	`GET /social-links; GET /clients/:userId`. Persistent entities: Profile, SocialLink.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-20.02 Add Social Link

UC ID and Name	UC-20.02 Add Social Link
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer
Trigger:	The actor selects Add Social Link from /account-profile (Social Links); /client/profile; public profile.
Description:	Validate the platform and external URL, then insert one link on the authenticated user profile.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Validate the platform and external URL, then insert one link on the authenticated user profile.

Normal Flow:	1. The actor opens /account-profile (Social Links); /client/profile; public profile and selects Add Social Link. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Validate the platform and external URL, then insert one link on the authenticated user profile. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-20-01:** At most one link per platform per profile is allowed, including concurrent submissions. **BR-NH-20-02:** Only the owning user can add or hard-delete a social link. **BR-NH-20-03:** Open approved external links in a new tab with rel="noopener noreferrer". **BR-NH-20-04:** Profile.id links the SocialLink table; the authenticated User.id resolves that profile.
Other Information:	`POST /social-links`. Persistent entities: Profile, SocialLink.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-20.03 Delete Social Link

UC ID and Name	UC-20.03 Delete Social Link
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer
Trigger:	The actor selects Delete Social Link from /account-profile (Social Links); /client/profile; public profile.
Description:	Hard-delete the selected owned social link after confirmation.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Hard-delete the selected owned social link after confirmation.
Normal Flow:	1. The actor opens /account-profile (Social Links); /client/profile; public profile and selects Delete Social Link. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Hard-delete the selected owned social link after confirmation. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-20-01:** At most one link per platform per profile is allowed, including concurrent submissions. **BR-NH-20-02:** Only the owning user can add or hard-delete a social link. **BR-NH-20-03:** Open approved external links in a new tab with rel="noopener noreferrer". **BR-NH-20-04:** Profile.id links the SocialLink table; the authenticated User.id resolves that profile.
Other Information:	`DELETE /social-links/:id`. Persistent entities: Profile, SocialLink.

Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

1.1.1.7 Favorite Freelancers

UC-21.01 View Favorite Freelancer List

UC ID and Name	UC-21.01 View Favorite Freelancer List
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client
Trigger:	The actor selects View Favorite Freelancer List from /client/profile (Favorites); /profiles/[id].
Description:	Load the current Client favorites and join public freelancer details.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Authorized data is displayed without modifying the underlying records.
Normal Flow:	1. The actor opens /client/profile (Favorites); /profiles/[id] and selects View Favorite Freelancer List. 2. The application submits the selected identifier or list filters. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Load the current Client favorites and join public freelancer details. 5. The screen renders the response, or an explicit empty state for a valid empty collection.
Alternative Flows:	No matching records: display an empty state. A failed request offers Retry.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **A1:** A valid empty collection displays an empty state; it must not be treated as a missing account. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.

Business Rules:	**BR-NH-21-01:** Only an authenticated Client manages their own favorites. **BR-NH-21-02:** Reject self-favoriting and targets without an active Freelancer role. **BR-NH-21-03:** The composite key (clientId, freelancerId) prevents duplicate relationships. **BR-NH-21-04:** Removal hard-deletes only the relationship; no freelancer account or profile is deleted. **BR-NH-21-05:** A successful add must be visible after list reload, including a new freelancer with minimal professional data.
Other Information:	`GET /favorites/freelancers`. Persistent entities: User, Profile, FreelancerProfile, FavoriteFreelancer.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-21.02 Add Freelancer to Favorites

UC ID and Name	UC-21.02 Add Freelancer to Favorites
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client
Trigger:	The actor selects Add Freelancer to Favorites from /client/profile (Favorites); /profiles/[id].
Description:	Insert a unique relationship from the current Client to an active freelancer.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Insert a unique relationship from the current Client to an active freelancer.
Normal Flow:	1. The actor opens /client/profile (Favorites); /profiles/[id] and selects Add Freelancer to Favorites. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Insert a unique relationship from the current Client to an active freelancer. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-21-01:** Only an authenticated Client manages their own favorites. **BR-NH-21-02:** Reject self-favoriting and targets without an active Freelancer role. **BR-NH-21-03:** The composite key (clientId, freelancerId) prevents duplicate relationships. **BR-NH-21-04:** Removal hard-deletes only the relationship; no freelancer account or profile is deleted. **BR-NH-21-05:** A successful add must be visible after list reload, including a new freelancer with minimal professional data.
Other Information:	`POST /favorites/freelancers/:freelancerId`. Persistent entities: User, Profile, FreelancerProfile, FavoriteFreelancer.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-21.03 Remove Freelancer from Favorites

UC ID and Name	UC-21.03 Remove Freelancer from Favorites
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client
Trigger:	The actor selects Remove Freelancer from Favorites from /client/profile (Favorites); /profiles/[id].
Description:	Hard-delete the current Client relationship and update the favorite toggle and list.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.

Post-conditions:	Hard-delete the current Client relationship and update the favorite toggle and list.
Normal Flow:	1. The actor opens /client/profile (Favorites); /profiles/[id] and selects Remove Freelancer from Favorites. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Hard-delete the current Client relationship and update the favorite toggle and list. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-21-01:** Only an authenticated Client manages their own favorites. **BR-NH-21-02:** Reject self-favoriting and targets without an active Freelancer role. **BR-NH-21-03:** The composite key (clientId, freelancerId) prevents duplicate relationships. **BR-NH-21-04:** Removal hard-deletes only the relationship; no freelancer account or profile is deleted. **BR-NH-21-05:** A successful add must be visible after list reload, including a new freelancer with minimal professional data.
Other Information:	`DELETE /favorites/freelancers/:freelancerId`. Persistent entities: User, Profile, FreelancerProfile, FavoriteFreelancer.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

1.1.1.8 Following Management

UC-29.01 View Following List

UC ID and Name	UC-29.01 View Following List
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client
Trigger:	The actor selects View Following List from /client/profile (Following); /profiles/[id].
Description:	Load freelancer relationships owned by the current Client and show the following state.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Authorized data is displayed without modifying the underlying records.
Normal Flow:	1. The actor opens /client/profile (Following); /profiles/[id] and selects View Following List. 2. The application submits the selected identifier or list filters. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Load freelancer relationships owned by the current Client and show the following state. 5. The screen renders the response, or an explicit empty state for a valid empty collection.
Alternative Flows:	No matching records: display an empty state. A failed request offers Retry.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **A1:** A valid empty collection displays an empty state; it must not be treated as a missing account. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-29-01:** Only a Client follows or unfollows an active freelancer; self-follow is rejected. **BR-NH-29-02:** A unique (clientId, freelancerId) mapping prevents duplicates. **BR-NH-29-03:** Create the follow and its recipient notification together in a transaction. **BR-NH-29-04:** Unfollow hard-deletes the mapping without deleting either user.

Other Information:	`GET /following/freelancers`. Persistent entities: User, FollowFreelancer, Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-29.02 Follow Freelancer

UC ID and Name	UC-29.02 Follow Freelancer
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client
Trigger:	The actor selects Follow Freelancer from /client/profile (Following); /profiles/[id].
Description:	Create a unique follow mapping and notify the target freelancer.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Create a unique follow mapping and notify the target freelancer.
Normal Flow:	1. The actor opens /client/profile (Following); /profiles/[id] and selects Follow Freelancer. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Create a unique follow mapping and notify the target freelancer. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.

Business Rules:	**BR-NH-29-01:** Only a Client follows or unfollows an active freelancer; self-follow is rejected. **BR-NH-29-02:** A unique (clientId, freelancerId) mapping prevents duplicates. **BR-NH-29-03:** Create the follow and its recipient notification together in a transaction. **BR-NH-29-04:** Unfollow hard-deletes the mapping without deleting either user.
Other Information:	`POST /following/freelancers/:freelancerId`. Persistent entities: User, FollowFreelancer, Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-29.03 Unfollow Freelancer

UC ID and Name	UC-29.03 Unfollow Freelancer
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client
Trigger:	The actor selects Unfollow Freelancer from /client/profile (Following); /profiles/[id].
Description:	Remove only the current Client follow relationship.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Remove only the current Client follow relationship.
Normal Flow:	1. The actor opens /client/profile (Following); /profiles/[id] and selects Unfollow Freelancer. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Remove only the current Client follow relationship. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.

Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-29-01:** Only a Client follows or unfollows an active freelancer; self-follow is rejected. **BR-NH-29-02:** A unique (clientId, freelancerId) mapping prevents duplicates. **BR-NH-29-03:** Create the follow and its recipient notification together in a transaction. **BR-NH-29-04:** Unfollow hard-deletes the mapping without deleting either user.
Other Information:	`DELETE /following/freelancers/:freelancerId`. Persistent entities: User, FollowFreelancer, Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

1.1.1.9 Notification Management

UC-30.01 View Notification List

UC ID and Name	UC-30.01 View Notification List
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer, Expert, Admin
Trigger:	The actor selects View Notification List from Header notification menu; /notifications.
Description:	Load the recipient active notifications in reverse chronological order.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Authorized data is displayed without modifying the underlying records.

Normal Flow:	1. The actor opens Header notification menu; /notifications and selects View Notification List. 2. The application submits the selected identifier or list filters. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Load the recipient active notifications in reverse chronological order. 5. The screen renders the response, or an explicit empty state for a valid empty collection.
Alternative Flows:	No matching records: display an empty state. A failed request offers Retry.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **A1:** A valid empty collection displays an empty state; it must not be treated as a missing account. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-30-01:** A user reads and changes only their own notifications. **BR-NH-30-02:** Exclude rows with deletedAt set and sort by createdAt descending. **BR-NH-30-03:** Mark-read sets isRead=true; mark-all affects only the recipient unread active rows. **BR-NH-30-04:** Deletion is a soft delete and updates the displayed list and unread count.
Other Information:	`GET /notifications`. Persistent entities: Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-30.02 Mark Notification as Read

UC ID and Name	UC-30.02 Mark Notification as Read
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer, Expert, Admin
Trigger:	The actor selects Mark Notification as Read from Header notification menu; /notifications.

Description:	Set isRead to true on one owned notification.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Set isRead to true on one owned notification.
Normal Flow:	1. The actor opens Header notification menu; /notifications and selects Mark Notification as Read. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Set isRead to true on one owned notification. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-30-01:** A user reads and changes only their own notifications. **BR-NH-30-02:** Exclude rows with deletedAt set and sort by createdAt descending. **BR-NH-30-03:** Mark-read sets isRead=true; mark-all affects only the recipient unread active rows. **BR-NH-30-04:** Deletion is a soft delete and updates the displayed list and unread count.
Other Information:	`PATCH /notifications/:id/read`. Persistent entities: Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-30.03 Mark All Notifications as Read

UC ID and Name	UC-30.03 Mark All Notifications as Read
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer, Expert, Admin
Trigger:	The actor selects Mark All Notifications as Read from Header notification menu; /notifications.
Description:	Update the current recipient unread, non-deleted notifications together.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Update the current recipient unread, non-deleted notifications together.
Normal Flow:	1. The actor opens Header notification menu; /notifications and selects Mark All Notifications as Read. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Update the current recipient unread, non-deleted notifications together. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-30-01:** A user reads and changes only their own notifications. **BR-NH-30-02:** Exclude rows with deletedAt set and sort by createdAt descending. **BR-NH-30-03:** Mark-read sets isRead=true; mark-all affects only the recipient unread active rows. **BR-NH-30-04:** Deletion is a soft delete and updates the displayed list and unread count.

Other Information:	`PATCH /notifications/read-all`. Persistent entities: Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-30.04 Delete Notification

UC ID and Name	UC-30.04 Delete Notification
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer, Expert, Admin
Trigger:	The actor selects Delete Notification from Header notification menu; /notifications.
Description:	Set deletedAt on the owned notification and hide it from the list.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Set deletedAt on the owned notification and hide it from the list.
Normal Flow:	1. The actor opens Header notification menu; /notifications and selects Delete Notification. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Set deletedAt on the owned notification and hide it from the list. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.

Business Rules:	**BR-NH-30-01:** A user reads and changes only their own notifications. **BR-NH-30-02:** Exclude rows with deletedAt set and sort by createdAt descending. **BR-NH-30-03:** Mark-read sets isRead=true; mark-all affects only the recipient unread active rows. **BR-NH-30-04:** Deletion is a soft delete and updates the displayed list and unread count.
Other Information:	`DELETE /notifications/:id`. Persistent entities: Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

1.1.1.10 Role Switching and Enrollment

UC-31.01 Switch Role

UC ID and Name	UC-31.01 Switch Role
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer
Trigger:	The actor selects Switch Role from Header role menu; role enrollment dialog.
Description:	Change the primary role to an already assigned role and issue updated authentication tokens.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Change the primary role to an already assigned role and issue updated authentication tokens.
Normal Flow:	1. The actor opens Header role menu; role enrollment dialog and selects Switch Role. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Change the primary role to an already assigned role and issue updated authentication tokens. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	High
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-31-01:** Switch only to a role already assigned to the authenticated user. **BR-NH-31-02:** Self-enrollment permits only Client or Freelancer; reject unsupported roles and duplicate UserRole mappings. **BR-NH-31-03:** Maintain exactly one active primary role for an account; role-context changes issue updated session tokens. **BR-NH-31-04:** Switching roles does not create a second user account or grant access to other users records.
Other Information:	`POST /auth/switch-role`. Persistent entities: User, Role, UserRole, Session.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-31.02 Join New Role

UC ID and Name	UC-31.02 Join New Role
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer
Trigger:	The actor selects Join New Role from Header role menu; role enrollment dialog.
Description:	Enroll in an allowed new role without duplicating the account or granting administrator privileges.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Enroll in an allowed new role without duplicating the account or granting administrator privileges.

Normal Flow:	1. The actor opens Header role menu; role enrollment dialog and selects Join New Role. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Enroll in an allowed new role without duplicating the account or granting administrator privileges. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	High
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-31-01:** Switch only to a role already assigned to the authenticated user. **BR-NH-31-02:** Self-enrollment permits only Client or Freelancer; reject unsupported roles and duplicate UserRole mappings. **BR-NH-31-03:** Maintain exactly one active primary role for an account; role-context changes issue updated session tokens. **BR-NH-31-04:** Switching roles does not create a second user account or grant access to other users records.
Other Information:	`POST /auth/join-role`. Persistent entities: User, Role, UserRole, Session.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

1.1.1.11 Contract Review Management

UC-39.01 View Review List

UC ID and Name	UC-39.01 View Review List
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer
Trigger:	The actor selects View Review List from /client/contracts/[slug]; /freelancer/contracts/[slug].
Description:	Retrieve reviews and active responses for a contract after verifying participant access.
Preconditions:	The actor is authenticated and has the required role and resource permission. The authenticated user is an authorized participant in the referenced contract.
Post-conditions:	Authorized data is displayed without modifying the underlying records.
Normal Flow:	1. The actor opens /client/contracts/[slug]; /freelancer/contracts/[slug] and selects View Review List. 2. The application submits the selected identifier or list filters. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Retrieve reviews and active responses for a contract after verifying participant access. 5. The screen renders the response, or an explicit empty state for a valid empty collection.
Alternative Flows:	No matching records: display an empty state. A failed request offers Retry.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **A1:** A valid empty collection displays an empty state; it must not be treated as a missing account. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-39-01:** Only contract participants can list or view contract reviews. **BR-NH-39-02:** A review can be created only after the contract is COMPLETED, for the other party. **BR-NH-39-03:** The pair (contractId, reviewerId) is unique. **BR-NH-39-04:** Only the review author edits or soft-deletes their review. **BR-NH-39-05:** An update must contain at least one accepted field. The current schema does not require a non-empty comment or an integer overall rating.

Other Information:	`GET /reviews/contracts/:contractId`. Persistent entities: Contract, Review, Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-39.02 View Review Detail

UC ID and Name	UC-39.02 View Review Detail
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer
Trigger:	The actor selects View Review Detail from /client/contracts/[slug]; /freelancer/contracts/[slug].
Description:	Retrieve one review and response after checking contract participation.
Preconditions:	The actor is authenticated and has the required role and resource permission. The authenticated user is an authorized participant in the referenced contract.
Post-conditions:	Authorized data is displayed without modifying the underlying records.
Normal Flow:	1. The actor opens /client/contracts/[slug]; /freelancer/contracts/[slug] and selects View Review Detail. 2. The application submits the selected identifier or list filters. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Retrieve one review and response after checking contract participation. 5. The screen renders the response, or an explicit empty state for a valid empty collection.
Alternative Flows:	No matching records: display an empty state. A failed request offers Retry.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **A1:** A valid empty collection displays an empty state; it must not be treated as a missing account. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.

Business Rules:	**BR-NH-39-01:** Only contract participants can list or view contract reviews. **BR-NH-39-02:** A review can be created only after the contract is COMPLETED, for the other party. **BR-NH-39-03:** The pair (contractId, reviewerId) is unique. **BR-NH-39-04:** Only the review author edits or soft-deletes their review. **BR-NH-39-05:** An update must contain at least one accepted field. The current schema does not require a non-empty comment or an integer overall rating.
Other Information:	`GET /reviews/:reviewId`. Persistent entities: Contract, Review, Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-39.03 Write Review

UC ID and Name	UC-39.03 Write Review
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer
Trigger:	The actor selects Write Review from /client/contracts/[slug]; /freelancer/contracts/[slug].
Description:	Create one review of the other participant in a completed contract and notify that participant.
Preconditions:	The actor is authenticated and has the required role and resource permission. The authenticated user is an authorized participant in the referenced contract.
Post-conditions:	Create one review of the other participant in a completed contract and notify that participant.
Normal Flow:	1. The actor opens /client/contracts/[slug]; /freelancer/contracts/[slug] and selects Write Review. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Create one review of the other participant in a completed contract and notify that participant. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-39-01:** Only contract participants can list or view contract reviews. **BR-NH-39-02:** A review can be created only after the contract is COMPLETED, for the other party. **BR-NH-39-03:** The pair (contractId, reviewerId) is unique. **BR-NH-39-04:** Only the review author edits or soft-deletes their review. **BR-NH-39-05:** An update must contain at least one accepted field. The current schema does not require a non-empty comment or an integer overall rating.
Other Information:	`POST /reviews/contracts/:contractId`. Persistent entities: Contract, Review, Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-39.04 Update Review

UC ID and Name	UC-39.04 Update Review
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer
Trigger:	The actor selects Update Review from /client/contracts/[slug]; /freelancer/contracts/[slug].
Description:	Update accepted fields of an existing review authored by the current user.
Preconditions:	The actor is authenticated and has the required role and resource permission. The authenticated user is an authorized participant in the referenced contract.

Post-conditions:	Update accepted fields of an existing review authored by the current user.
Normal Flow:	1. The actor opens /client/contracts/[slug]; /freelancer/contracts/[slug] and selects Update Review. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Update accepted fields of an existing review authored by the current user. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-39-01:** Only contract participants can list or view contract reviews. **BR-NH-39-02:** A review can be created only after the contract is COMPLETED, for the other party. **BR-NH-39-03:** The pair (contractId, reviewerId) is unique. **BR-NH-39-04:** Only the review author edits or soft-deletes their review. **BR-NH-39-05:** An update must contain at least one accepted field. The current schema does not require a non-empty comment or an integer overall rating.
Other Information:	`PATCH /reviews/:reviewId`. Persistent entities: Contract, Review, Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-39.05 Delete Review

UC ID and Name	UC-39.05 Delete Review
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer
Trigger:	The actor selects Delete Review from /client/contracts/[slug]; /freelancer/contracts/[slug].
Description:	Soft-delete the owned review so it no longer appears as active feedback.
Preconditions:	The actor is authenticated and has the required role and resource permission. The authenticated user is an authorized participant in the referenced contract.
Post-conditions:	Soft-delete the owned review so it no longer appears as active feedback.
Normal Flow:	1. The actor opens /client/contracts/[slug]; /freelancer/contracts/[slug] and selects Delete Review. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Soft-delete the owned review so it no longer appears as active feedback. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.

Business Rules:	**BR-NH-39-01:** Only contract participants can list or view contract reviews. **BR-NH-39-02:** A review can be created only after the contract is COMPLETED, for the other party. **BR-NH-39-03:** The pair (contractId, reviewerId) is unique. **BR-NH-39-04:** Only the review author edits or soft-deletes their review. **BR-NH-39-05:** An update must contain at least one accepted field. The current schema does not require a non-empty comment or an integer overall rating.
Other Information:	`DELETE /reviews/:reviewId`. Persistent entities: Contract, Review, Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

1.1.1.12 Review Response Management

UC-40.01 Respond to Review

UC ID and Name	UC-40.01 Respond to Review
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer
Trigger:	The actor selects Respond to Review from Contract Reviews section.
Description:	Create an active response from the reviewee and notify the review author.
Preconditions:	The actor is authenticated and has the required role and resource permission. The authenticated user is an authorized participant in the referenced contract.
Post-conditions:	Create an active response from the reviewee and notify the review author.

Normal Flow:	1. The actor opens Contract Reviews section and selects Respond to Review. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Create an active response from the reviewee and notify the review author. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-40-01:** Only the user being reviewed may respond. **BR-NH-40-02:** There is at most one active response per review. **BR-NH-40-03:** Only the response author edits or soft-deletes it. **BR-NH-40-04:** Soft-deleted responses must not be rendered as active feedback.
Other Information:	`POST /reviews/:reviewId/response`. Persistent entities: Review, ReviewResponse, Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-40.02 Update Review Response

UC ID and Name	UC-40.02 Update Review Response
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer
Trigger:	The actor selects Update Review Response from Contract Reviews section.

Description:	Save validated text in the current user response.
Preconditions:	The actor is authenticated and has the required role and resource permission. The authenticated user is an authorized participant in the referenced contract.
Post-conditions:	Save validated text in the current user response.
Normal Flow:	1. The actor opens Contract Reviews section and selects Update Review Response. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Save validated text in the current user response. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-40-01:** Only the user being reviewed may respond. **BR-NH-40-02:** There is at most one active response per review. **BR-NH-40-03:** Only the response author edits or soft-deletes it. **BR-NH-40-04:** Soft-deleted responses must not be rendered as active feedback.
Other Information:	`PATCH /reviews/responses/:responseId`. Persistent entities: Review, ReviewResponse, Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-40.03 Delete Review Response

UC ID and Name	UC-40.03 Delete Review Response
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer
Trigger:	The actor selects Delete Review Response from Contract Reviews section.
Description:	Soft-delete the response authored by the current user.
Preconditions:	The actor is authenticated and has the required role and resource permission. The authenticated user is an authorized participant in the referenced contract.
Post-conditions:	Soft-delete the response authored by the current user.
Normal Flow:	1. The actor opens Contract Reviews section and selects Delete Review Response. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Soft-delete the response authored by the current user. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-40-01:** Only the user being reviewed may respond. **BR-NH-40-02:** There is at most one active response per review. **BR-NH-40-03:** Only the response author edits or soft-deletes it. **BR-NH-40-04:** Soft-deleted responses must not be rendered as active feedback.
Other Information:	`DELETE /reviews/responses/:responseId`. Persistent entities: Review, ReviewResponse, Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

1.1.1.13 General Account Profile

UC-41.01 View Profile

UC ID and Name	UC-41.01 View Profile
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer, Expert, Admin
Trigger:	The actor selects View Profile from /account-profile (General Settings).
Description:	Read the authenticated account profile, email and assigned roles.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Authorized data is displayed without modifying the underlying records.
Normal Flow:	1. The actor opens /account-profile (General Settings) and selects View Profile. 2. The application submits the selected identifier or list filters. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Read the authenticated account profile, email and assigned roles. 5. The screen renders the response, or an explicit empty state for a valid empty collection.
Alternative Flows:	No matching records: display an empty state. A failed request offers Retry.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **A1:** A valid empty collection displays an empty state; it must not be treated as a missing account. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.

Business Rules:	**BR-NH-41-01:** The server resolves the account from the session; users cannot edit another account via body IDs. **BR-NH-41-02:** Verify the current password before hashing and storing a replacement; do not log credentials. **BR-NH-41-03:** Apply the profile revision policy to general profile changes. **BR-NH-41-04:** Avatar files must be validated; missing stored files return 404 without terminating the API process.
Other Information:	`GET /account-profile`. Persistent entities: User, Profile, ProfileRevision.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-41.02 Update Profile

UC ID and Name	UC-41.02 Update Profile
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer, Expert, Admin
Trigger:	The actor selects Update Profile from /account-profile (General Settings).
Description:	Save or submit general display name and biography according to profile strength.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Save or submit general display name and biography according to profile strength.
Normal Flow:	1. The actor opens /account-profile (General Settings) and selects Update Profile. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Save or submit general display name and biography according to profile strength. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.

Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-41-01:** The server resolves the account from the session; users cannot edit another account via body IDs. **BR-NH-41-02:** Verify the current password before hashing and storing a replacement; do not log credentials. **BR-NH-41-03:** Apply the profile revision policy to general profile changes. **BR-NH-41-04:** Avatar files must be validated; missing stored files return 404 without terminating the API process.
Other Information:	`PUT /account-profile`. Persistent entities: User, Profile, ProfileRevision.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-41.03 Change Password

UC ID and Name	UC-41.03 Change Password
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer, Expert, Admin
Trigger:	The actor selects Change Password from /account-profile (General Settings).
Description:	Verify the current credential, hash the confirmed replacement and update the password.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Verify the current credential, hash the confirmed replacement and update the password.

Normal Flow:	1. The actor opens /account-profile (General Settings) and selects Change Password. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Verify the current credential, hash the confirmed replacement and update the password. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-41-01:** The server resolves the account from the session; users cannot edit another account via body IDs. **BR-NH-41-02:** Verify the current password before hashing and storing a replacement; do not log credentials. **BR-NH-41-03:** Apply the profile revision policy to general profile changes. **BR-NH-41-04:** Avatar files must be validated; missing stored files return 404 without terminating the API process.
Other Information:	`PUT /account-profile/password`. Persistent entities: User, Profile, ProfileRevision.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-41.04 Upload Avatar

UC ID and Name	UC-41.04 Upload Avatar
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Client, Freelancer, Expert, Admin
Trigger:	The actor selects Upload Avatar from /account-profile (General Settings).
Description:	Validate image content and size, store it, and update the owned avatar URL.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Validate image content and size, store it, and update the owned avatar URL.
Normal Flow:	1. The actor opens /account-profile (General Settings) and selects Upload Avatar. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Validate image content and size, store it, and update the owned avatar URL. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-41-01:** The server resolves the account from the session; users cannot edit another account via body IDs. **BR-NH-41-02:** Verify the current password before hashing and storing a replacement; do not log credentials. **BR-NH-41-03:** Apply the profile revision policy to general profile changes. **BR-NH-41-04:** Avatar files must be validated; missing stored files return 404 without terminating the API process.

Other Information:	`POST /account-profile/avatar`. Persistent entities: User, Profile, ProfileRevision.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

1.1.1.14 Profile Revision Review and Expert Profile

UC-47.01 View Pending Profile Update Request List

UC ID and Name	UC-47.01 View Pending Profile Update Request List
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Admin
Trigger:	The actor selects View Pending Profile Update Request List from /admin/profile-revisions; /expert/profile.
Description:	Filter and paginate revisions, defaulting the review workflow to pending requests.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Authorized data is displayed without modifying the underlying records.
Normal Flow:	1. The actor opens /admin/profile-revisions; /expert/profile and selects View Pending Profile Update Request List. 2. The application submits the selected identifier or list filters. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Filter and paginate revisions, defaulting the review workflow to pending requests. 5. The screen renders the response, or an explicit empty state for a valid empty collection.
Alternative Flows:	No matching records: display an empty state. A failed request offers Retry.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **A1:** A valid empty collection displays an empty state; it must not be treated as a missing account. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Priority:	High
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-47-01:** Only an authorized administrator can inspect or decide profile revisions. **BR-NH-47-02:** Only PENDING revisions may be approved or rejected. Decision and profile writes are transactional. **BR-NH-47-03:** Approval applies validated proposed data to the correct subtype; rejection retains existing public data. **BR-NH-47-04:** Record reviewer, decision time and notes; preserve history and notify the requester. **BR-NH-47-05:** An Expert submits their own update as a pending revision. Administrative expert editing is a separate authorized operation.
Other Information:	`GET /admin/profile-revisions`. Persistent entities: ProfileRevision, Profile, FreelancerProfile, ClientProfile, Expert, Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-47.02 View Profile Update Request Detail

UC ID and Name	UC-47.02 View Profile Update Request Detail
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Admin
Trigger:	The actor selects View Profile Update Request Detail from /admin/profile-revisions; /expert/profile.
Description:	Display the current and proposed snapshots, requester, profile type and prior review notes.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Authorized data is displayed without modifying the underlying records.

Normal Flow:	1. The actor opens /admin/profile-revisions; /expert/profile and selects View Profile Update Request Detail. 2. The application submits the selected identifier or list filters. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Display the current and proposed snapshots, requester, profile type and prior review notes. 5. The screen renders the response, or an explicit empty state for a valid empty collection.
Alternative Flows:	No matching records: display an empty state. A failed request offers Retry.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **A1:** A valid empty collection displays an empty state; it must not be treated as a missing account. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	High
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-47-01:** Only an authorized administrator can inspect or decide profile revisions. **BR-NH-47-02:** Only PENDING revisions may be approved or rejected. Decision and profile writes are transactional. **BR-NH-47-03:** Approval applies validated proposed data to the correct subtype; rejection retains existing public data. **BR-NH-47-04:** Record reviewer, decision time and notes; preserve history and notify the requester. **BR-NH-47-05:** An Expert submits their own update as a pending revision. Administrative expert editing is a separate authorized operation.
Other Information:	`GET /admin/profile-revisions/:id`. Persistent entities: ProfileRevision, Profile, FreelancerProfile, ClientProfile, Expert, Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-47.03 Approve Profile Update Request

UC ID and Name	UC-47.03 Approve Profile Update Request
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Admin
Trigger:	The actor selects Approve Profile Update Request from /admin/profile-revisions; /expert/profile.
Description:	Apply validated proposed changes and record an APPROVED decision atomically.
Preconditions:	The actor is authenticated and has the required role and resource permission. The selected revision exists and is still PENDING.
Post-conditions:	Apply validated proposed changes and record an APPROVED decision atomically.
Normal Flow:	1. The actor opens /admin/profile-revisions; /expert/profile and selects Approve Profile Update Request. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Apply validated proposed changes and record an APPROVED decision atomically. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	High
Frequency of Use:	As required by the actor workflow.

Business Rules:	**BR-NH-47-01:** Only an authorized administrator can inspect or decide profile revisions. **BR-NH-47-02:** Only PENDING revisions may be approved or rejected. Decision and profile writes are transactional. **BR-NH-47-03:** Approval applies validated proposed data to the correct subtype; rejection retains existing public data. **BR-NH-47-04:** Record reviewer, decision time and notes; preserve history and notify the requester. **BR-NH-47-05:** An Expert submits their own update as a pending revision. Administrative expert editing is a separate authorized operation.
Other Information:	`PATCH /admin/profile-revisions/:id/approve`. Persistent entities: ProfileRevision, Profile, FreelancerProfile, ClientProfile, Expert, Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-47.04 Reject Profile Update Request

UC ID and Name	UC-47.04 Reject Profile Update Request
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Admin
Trigger:	The actor selects Reject Profile Update Request from /admin/profile-revisions; /expert/profile.
Description:	Keep approved profile data unchanged and record REJECTED with an explanatory note.
Preconditions:	The actor is authenticated and has the required role and resource permission. The selected revision exists and is still PENDING.
Post-conditions:	Keep approved profile data unchanged and record REJECTED with an explanatory note.
Normal Flow:	1. The actor opens /admin/profile-revisions; /expert/profile and selects Reject Profile Update Request. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Keep approved profile data unchanged and record REJECTED with an explanatory note. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	High
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-47-01:** Only an authorized administrator can inspect or decide profile revisions. **BR-NH-47-02:** Only PENDING revisions may be approved or rejected. Decision and profile writes are transactional. **BR-NH-47-03:** Approval applies validated proposed data to the correct subtype; rejection retains existing public data. **BR-NH-47-04:** Record reviewer, decision time and notes; preserve history and notify the requester. **BR-NH-47-05:** An Expert submits their own update as a pending revision. Administrative expert editing is a separate authorized operation.
Other Information:	`PATCH /admin/profile-revisions/:id/reject`. Persistent entities: ProfileRevision, Profile, FreelancerProfile, ClientProfile, Expert, Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-47.05 View Profile Update Request History

UC ID and Name	UC-47.05 View Profile Update Request History
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Admin
Trigger:	The actor selects View Profile Update Request History from /admin/profile-revisions; /expert/profile.
Description:	Allow Admin to filter historical decisions; a requester can read only their latest revision for a profile type.

Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Authorized data is displayed without modifying the underlying records.
Normal Flow:	1. The actor opens /admin/profile-revisions; /expert/profile and selects View Profile Update Request History. 2. The application submits the selected identifier or list filters. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Allow Admin to filter historical decisions; a requester can read only their latest revision for a profile type. 5. The screen renders the response, or an explicit empty state for a valid empty collection.
Alternative Flows:	No matching records: display an empty state. A failed request offers Retry.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **A1:** A valid empty collection displays an empty state; it must not be treated as a missing account. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	High
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-47-01:** Only an authorized administrator can inspect or decide profile revisions. **BR-NH-47-02:** Only PENDING revisions may be approved or rejected. Decision and profile writes are transactional. **BR-NH-47-03:** Approval applies validated proposed data to the correct subtype; rejection retains existing public data. **BR-NH-47-04:** Record reviewer, decision time and notes; preserve history and notify the requester. **BR-NH-47-05:** An Expert submits their own update as a pending revision. Administrative expert editing is a separate authorized operation.
Other Information:	`GET /admin/profile-revisions; GET /profile-revisions/me?type=...` Persistent entities: ProfileRevision, Profile, FreelancerProfile, ClientProfile, Expert, Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-47.06 Update Profile Expert

UC ID and Name	UC-47.06 Update Profile Expert
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Expert
Trigger:	The actor selects Update Profile Expert from /admin/profile-revisions; /expert/profile.
Description:	Submit the owned Expert profile as a pending revision for administrator review.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Submit the owned Expert profile as a pending revision for administrator review.
Normal Flow:	1. The actor opens /admin/profile-revisions; /expert/profile and selects Update Profile Expert. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Submit the owned Expert profile as a pending revision for administrator review. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	High
Frequency of Use:	As required by the actor workflow.

Business Rules:	**BR-NH-47-01:** Only an authorized administrator can inspect or decide profile revisions. **BR-NH-47-02:** Only PENDING revisions may be approved or rejected. Decision and profile writes are transactional. **BR-NH-47-03:** Approval applies validated proposed data to the correct subtype; rejection retains existing public data. **BR-NH-47-04:** Record reviewer, decision time and notes; preserve history and notify the requester. **BR-NH-47-05:** An Expert submits their own update as a pending revision. Administrative expert editing is a separate authorized operation.
Other Information:	`PUT /expert-profile/me`. Persistent entities: ProfileRevision, Profile, FreelancerProfile, ClientProfile, Expert, Notification.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

1.1.1.15 Freelancer CV Management

UC-07.03 Upload CV

UC ID and Name	UC-07.03 Upload CV
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Freelancer
Trigger:	The actor selects Upload CV from /freelancer/profile; /profiles/[id].
Description:	Validate a PDF up to 10 MB and attach it to the owned FreelancerProfile.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Validate a PDF up to 10 MB and attach it to the owned FreelancerProfile.

Normal Flow:	1. The actor opens /freelancer/profile; /profiles/[id] and selects Upload CV. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Validate a PDF up to 10 MB and attach it to the owned FreelancerProfile. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-CV-01:** Only the owning freelancer can upload, view, replace or delete their CV. **BR-NH-CV-02:** Uploads are PDF-only, at most 10 MB, and must have a valid PDF signature; reject missing, oversized or invalid files. **BR-NH-CV-03:** Store CV URL, file name, public ID and upload time on FreelancerProfile. Do not expose private storage paths in public profile responses. **BR-NH-CV-04:** A replacement updates metadata only after the new file is stored. A missing local file returns 404 without crashing the API. **BR-NH-CV-05:** Deleting a CV clears its metadata and triggers a search-index update; it does not delete the profile or previously confirmed professional sections.
Other Information:	`POST /profiles/:id/cv`. Persistent entities: Profile, FreelancerProfile, ProfileRevision.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-07.04 View CV

UC ID and Name	UC-07.04 View CV
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Freelancer
Trigger:	The actor selects View CV from /freelancer/profile; /profiles/[id].
Description:	Authorize the owner and serve or redirect to the current CV; return 404 when its stored file is unavailable.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Authorized data is displayed without modifying the underlying records.
Normal Flow:	1. The actor opens /freelancer/profile; /profiles/[id] and selects View CV. 2. The application submits the selected identifier or list filters. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Authorize the owner and serve or redirect to the current CV; return 404 when its stored file is unavailable. 5. The screen renders the response, or an explicit empty state for a valid empty collection.
Alternative Flows:	No matching records: display an empty state. A failed request offers Retry.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **A1:** A valid empty collection displays an empty state; it must not be treated as a missing account. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.

Business Rules:	**BR-NH-CV-01:** Only the owning freelancer can upload, view, replace or delete their CV. **BR-NH-CV-02:** Uploads are PDF-only, at most 10 MB, and must have a valid PDF signature; reject missing, oversized or invalid files. **BR-NH-CV-03:** Store CV URL, file name, public ID and upload time on FreelancerProfile. Do not expose private storage paths in public profile responses. **BR-NH-CV-04:** A replacement updates metadata only after the new file is stored. A missing local file returns 404 without crashing the API. **BR-NH-CV-05:** Deleting a CV clears its metadata and triggers a search-index update; it does not delete the profile or previously confirmed professional sections.
Other Information:	`GET /profiles/:id/cv/file`. Persistent entities: Profile, FreelancerProfile, ProfileRevision.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-07.05 Replace CV

UC ID and Name	UC-07.05 Replace CV
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Freelancer
Trigger:	The actor selects Replace CV from /freelancer/profile; /profiles/[id].
Description:	Store a valid replacement PDF, update CV metadata and clean up the previous file after a successful update.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Store a valid replacement PDF, update CV metadata and clean up the previous file after a successful update.

Normal Flow:	1. The actor opens /freelancer/profile; /profiles/[id] and selects Replace CV. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Store a valid replacement PDF, update CV metadata and clean up the previous file after a successful update. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.
Business Rules:	**BR-NH-CV-01:** Only the owning freelancer can upload, view, replace or delete their CV. **BR-NH-CV-02:** Uploads are PDF-only, at most 10 MB, and must have a valid PDF signature; reject missing, oversized or invalid files. **BR-NH-CV-03:** Store CV URL, file name, public ID and upload time on FreelancerProfile. Do not expose private storage paths in public profile responses. **BR-NH-CV-04:** A replacement updates metadata only after the new file is stored. A missing local file returns 404 without crashing the API. **BR-NH-CV-05:** Deleting a CV clears its metadata and triggers a search-index update; it does not delete the profile or previously confirmed professional sections.
Other Information:	`POST /profiles/:id/cv`. Persistent entities: Profile, FreelancerProfile, ProfileRevision.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

UC-07.06 Delete CV

UC ID and Name	UC-07.06 Delete CV
Created By:	Nguyen Lu Nhat Huy; updated 08/11/2026
Primary Actor:	Freelancer
Trigger:	The actor selects Delete CV from /freelancer/profile; /profiles/[id].
Description:	Clear CV URL, public ID, file name and upload time, and update the related search index.
Preconditions:	The actor is authenticated and has the required role and resource permission. The relevant database and dependent services are available.
Post-conditions:	Clear CV URL, public ID, file name and upload time, and update the related search index.
Normal Flow:	1. The actor opens /freelancer/profile; /profiles/[id] and selects Delete CV. 2. The application presents the current values or action confirmation; the actor supplies the required data and submits. 3. The API checks authentication where required, authorization, resource existence and input validity. 4. Clear CV URL, public ID, file name and upload time, and update the related search index. 5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.
Alternative Flows:	Cancel before submission: discard unsaved input or close the confirmation without changing stored data.
Exceptions:	**E1:** Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data. **E2:** Missing, deleted or inaccessible resource: show a not-found result. **E3:** Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values. **E4:** Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
Priority:	Medium
Frequency of Use:	As required by the actor workflow.

Business Rules:	**BR-NH-CV-01:** Only the owning freelancer can upload, view, replace or delete their CV. **BR-NH-CV-02:** Uploads are PDF-only, at most 10 MB, and must have a valid PDF signature; reject missing, oversized or invalid files. **BR-NH-CV-03:** Store CV URL, file name, public ID and upload time on FreelancerProfile. Do not expose private storage paths in public profile responses. **BR-NH-CV-04:** A replacement updates metadata only after the new file is stored. A missing local file returns 404 without crashing the API. **BR-NH-CV-05:** Deleting a CV clears its metadata and triggers a search-index update; it does not delete the profile or previously confirmed professional sections.
Other Information:	`DELETE /profiles/:id/cv`. Persistent entities: Profile, FreelancerProfile, ProfileRevision.
Assumptions:	This specification defines the required behavior; implementation gaps are recorded separately and do not imply that every requirement has passed acceptance.

2. Functional Requirements

2.1 Freelancer Profile Management

2.1.1 Function: View Freelancer Profile (UC-07.01)

Function Trigger

The actor selects View Freelancer Profile. Navigation: /freelancer/profile; /profiles/[id].

Function Description

Actors / Roles

Guest, Client, Freelancer.

Purpose

Retrieve approved Profile and FreelancerProfile fields in one response.

Interface and Data Processing

`GET /profiles/:id`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Profile, FreelancerProfile, ProfileRevision. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

View Freelancer Profile
Profile header with name and availability; professional sections; Skills and Portfolio panels; owner-only Edit and CV controls.
Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Identifier	Positive resource ID from the route; ownership is checked by the API.

Validation and Business Rules

- Use Profile.id for profile routes; User.id identifies the owner and must not be substituted for the profile ID.
- Only the owner changes professional data. Public output excludes credentials and private CV locations.
- Omitted optional fields are preserved. An explicit null or empty list clears the corresponding value.
- Freelancer and Client updates below 20 percent profile strength require a pending revision; other updates apply directly. Pending changes are not shown as approved public data.

Normal Flow

1. The actor opens /freelancer/profile; /profiles/[id] and selects View Freelancer Profile.
2. The application submits the selected identifier or list filters.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Retrieve approved Profile and FreelancerProfile fields in one response.
5. The screen renders the response, or an explicit empty state for a valid empty collection.

Alternative Flow

Show an informative empty state for a valid empty result and allow retry after a transport failure.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
A1	A valid empty collection displays an empty state; it must not be treated as a missing account.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Retrieve approved Profile and FreelancerProfile fields in one response.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful view freelancer profile produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.1.2 Function: Update Freelancer Profile (UC-07.02)

Function Trigger

The actor selects Update Freelancer Profile. Navigation: /freelancer/profile; /profiles/[id].

Function Description

Actors / Roles

Freelancer.

Purpose

Persist basic and professional fields together or create a pending revision under the review policy.

Interface and Data Processing

`PUT /profiles/:id`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Profile, FreelancerProfile, ProfileRevision. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Update Freelancer Profile

Profile header with name and availability; professional sections; Skills and Portfolio panels; owner-only Edit and CV controls.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
displayName / title	Required trimmed text, 1–255 characters each.
bio	Optional text up to 5,000 characters; null clears it.
education / certifications / languages / experience	Each list has at most 20 non-empty entries, each up to 255 characters.
availabilityStatus	A supported availability enum value.

Validation and Business Rules

- Use Profile.id for profile routes; User.id identifies the owner and must not be substituted for the profile ID.
- Only the owner changes professional data. Public output excludes credentials and private CV locations.
- Omitted optional fields are preserved. An explicit null or empty list clears the corresponding value.
- Freelancer and Client updates below 20 percent profile strength require a pending revision; other updates apply directly. Pending changes are not shown as approved public data.

Normal Flow

1. The actor opens /freelancer/profile; /profiles/[id] and selects Update Freelancer Profile.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Persist basic and professional fields together or create a pending revision under the review policy.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Persist basic and professional fields together or create a pending revision under the review policy.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful update freelancer profile produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.2 Freelancer Skills Management

2.2.1 Function: View Freelancer Skill List (UC-08.01)

Function Trigger

The actor selects View Freelancer Skill List. Navigation: /profiles/[id]; /freelancer/profile.

Function Description

Actors / Roles

Guest, Client, Freelancer.

Purpose

Join FreelancerSkill with active Skill catalog entries and show proficiency levels.

Interface and Data Processing

`GET /profiles/:id/skills`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Profile, FreelancerProfile, FreelancerSkill, Skill. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

View Freelancer Skill List

Skill list with catalog name, proficiency /10 and owner-only remove action; Add Skill dialog with suggestions and level selector.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Selection	Current account or selected profile/contract; no pagination is assumed unless exposed by the endpoint.

Validation and Business Rules

- A freelancer can add and delete mappings only on their own profile.
- The pair (freelancerProfileId, skillId) must be unique, including concurrent requests.
- Deleting a skill permanently removes FreelancerSkill but retains the shared Skill definition.
- Show an empty state when there are no active skills.

Normal Flow

1. The actor opens /profiles/[id]; /freelancer/profile and selects View Freelancer Skill List.
2. The application submits the selected identifier or list filters.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Join FreelancerSkill with active Skill catalog entries and show proficiency levels.
5. The screen renders the response, or an explicit empty state for a valid empty collection.

Alternative Flow

Show an informative empty state for a valid empty result and allow retry after a transport failure.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.

Case	Description
A1	A valid empty collection displays an empty state; it must not be treated as a missing account.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Join FreelancerSkill with active Skill catalog entries and show proficiency levels.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful view freelancer skill list produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.2.2 Function: Add Freelancer Skill (UC-08.02)

Function Trigger

The actor selects Add Freelancer Skill. Navigation: /profiles/[id]; /freelancer/profile.

Function Description

Actors / Roles

Freelancer.

Purpose

Resolve or create the canonical catalog skill and insert one owned FreelancerSkill mapping.

Interface and Data Processing

`POST /profiles/:id/skills`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Profile, FreelancerProfile, FreelancerSkill, Skill. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Add Freelancer Skill

Skill list with catalog name, proficiency /10 and owner-only remove action; Add Skill dialog with suggestions and level selector.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
skillName	Required trimmed name up to 100 characters, entered or selected from suggestions.
proficiencyLevel	Required integer from 1 to 10.
id	Skill deletion uses FreelancerSkill.id, not Skill.id.

Validation and Business Rules

- A freelancer can add and delete mappings only on their own profile.
- The pair (freelancerProfileId, skillId) must be unique, including concurrent requests.
- Deleting a skill permanently removes FreelancerSkill but retains the shared Skill definition.
- Show an empty state when there are no active skills.

Normal Flow

1. The actor opens /profiles/[id]; /freelancer/profile and selects Add Freelancer Skill.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.

4. Resolve or create the canonical catalog skill and insert one owned FreelancerSkill mapping.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Resolve or create the canonical catalog skill and insert one owned FreelancerSkill mapping.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful add freelancer skill produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.2.3 Function: Delete Freelancer Skill (UC-08.03)

Function Trigger

The actor selects Delete Freelancer Skill. Navigation: /profiles/[id]; /freelancer/profile.

Function Description

Actors / Roles

Freelancer.

Purpose

Hard-delete the owned mapping while retaining its Skill catalog entry.

Interface and Data Processing

`DELETE /profiles/skills/:id`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Profile, FreelancerProfile, FreelancerSkill, Skill. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Delete Freelancer Skill

Skill list with catalog name, proficiency /10 and owner-only remove action; Add Skill dialog with suggestions and level selector.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Identifier	Positive resource ID from the route; ownership is checked by the API.

Validation and Business Rules

- A freelancer can add and delete mappings only on their own profile.
- The pair (freelancerProfileId, skillId) must be unique, including concurrent requests.
- Deleting a skill permanently removes FreelancerSkill but retains the shared Skill definition.
- Show an empty state when there are no active skills.

Normal Flow

1. The actor opens /profiles/[id]; /freelancer/profile and selects Delete Freelancer Skill.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Hard-delete the owned mapping while retaining its Skill catalog entry.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.

Case	Description
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Hard-delete the owned mapping while retaining its Skill catalog entry.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful delete freelancer skill produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.3 Portfolio Management

2.3.1 Function: View Portfolio List (UC-09.01)

Function Trigger

The actor selects View Portfolio List. Navigation: /profiles/[id]; /freelancer/profile.

Function Description

Actors / Roles

Guest, Client, Freelancer.

Purpose

List only non-deleted PortfolioItem records of the selected freelancer.

Interface and Data Processing

`GET /profiles/:id/portfolios`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Profile, FreelancerProfile, PortfolioItem. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

View Portfolio List

Portfolio card grid with title, description and technologies; detail dialog; owner-only Add, Edit and Delete controls; confirmation before deletion.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Selection	Current account or selected profile/contract; no pagination is assumed unless exposed by the endpoint.

Validation and Business Rules

- Only the owning freelancer creates, updates or deletes a portfolio item.
- Only items whose deletedAt is null are listed, opened or edited.
- Deletion sets deletedAt; it does not hard-delete the item.
- The agreed database scope is title, description, technologies and project URL; media gallery, uploads and project role are excluded.

Normal Flow

1. The actor opens /profiles/[id]; /freelancer/profile and selects View Portfolio List.
2. The application submits the selected identifier or list filters.

3. The API checks authentication where required, authorization, resource existence and input validity.
4. List only non-deleted PortfolioItem records of the selected freelancer.
5. The screen renders the response, or an explicit empty state for a valid empty collection.

Alternative Flow

Show an informative empty state for a valid empty result and allow retry after a transport failure.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
A1	A valid empty collection displays an empty state; it must not be treated as a missing account.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	List only non-deleted PortfolioItem records of the selected freelancer.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful view portfolio list produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.3.2 Function: Add Portfolio (UC-09.02)

Function Trigger

The actor selects Add Portfolio. Navigation: /profiles/[id]; /freelancer/profile.

Function Description

Actors / Roles

Freelancer.

Purpose

Create an owned portfolio item with title, description, technologies and project link.

Interface and Data Processing

`POST /profiles/:id/portfolios`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Profile, FreelancerProfile, PortfolioItem. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Add Portfolio

Portfolio card grid with title, description and technologies; detail dialog; owner-only Add, Edit and Delete controls; confirmation before deletion.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
title	Required trimmed title, 1–255 characters.
description	Optional text, up to 5,000 characters.
technologies	At most 20 non-empty names, each up to 100 characters.
projectUrl	Optional HTTP/HTTPS project link up to 500 characters.

Validation and Business Rules

- Only the owning freelancer creates, updates or deletes a portfolio item.
- Only items whose deletedAt is null are listed, opened or edited.
- Deletion sets deletedAt; it does not hard-delete the item.
- The agreed database scope is title, description, technologies and project URL; media gallery, uploads and project role are excluded.

Normal Flow

1. The actor opens /profiles/[id]; /freelancer/profile and selects Add Portfolio.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Create an owned portfolio item with title, description, technologies and project link.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Create an owned portfolio item with title, description, technologies and project link.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful add portfolio produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.3.3 Function: View Portfolio Detail (UC-09.03)

Function Trigger

The actor selects View Portfolio Detail. Navigation: /profiles/[id]; /freelancer/profile.

Function Description

Actors / Roles

Guest, Client, Freelancer.

Purpose

Display a single non-deleted portfolio item and its external project link.

Interface and Data Processing

`GET /profiles/portfolios/:id`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Profile, FreelancerProfile, PortfolioItem. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

View Portfolio Detail

Portfolio card grid with title, description and technologies; detail dialog; owner-only Add, Edit and Delete controls; confirmation before deletion.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Identifier	Positive resource ID from the route; ownership is checked by the API.

Validation and Business Rules

- Only the owning freelancer creates, updates or deletes a portfolio item.
- Only items whose deletedAt is null are listed, opened or edited.
- Deletion sets deletedAt; it does not hard-delete the item.
- The agreed database scope is title, description, technologies and project URL; media gallery, uploads and project role are excluded.

Normal Flow

1. The actor opens /profiles/[id]; /freelancer/profile and selects View Portfolio Detail.
2. The application submits the selected identifier or list filters.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Display a single non-deleted portfolio item and its external project link.
5. The screen renders the response, or an explicit empty state for a valid empty collection.

Alternative Flow

Show an informative empty state for a valid empty result and allow retry after a transport failure.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.

Case	Description
A1	A valid empty collection displays an empty state; it must not be treated as a missing account.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Display a single non-deleted portfolio item and its external project link.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful view portfolio detail produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.3.4 Function: Update Portfolio (UC-09.04)

Function Trigger

The actor selects Update Portfolio. Navigation: /profiles/[id]; /freelancer/profile.

Function Description

Actors / Roles

Freelancer.

Purpose

Save validated edits on an owned, non-deleted portfolio item.

Interface and Data Processing

`PUT /profiles/portfolios/:id`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Profile, FreelancerProfile, PortfolioItem. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Update Portfolio

Portfolio card grid with title, description and technologies; detail dialog; owner-only Add, Edit and Delete controls; confirmation before deletion.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
title	Required trimmed title, 1–255 characters.
description	Optional text, up to 5,000 characters.
technologies	At most 20 non-empty names, each up to 100 characters.
projectUrl	Optional HTTP/HTTPS project link up to 500 characters.

Validation and Business Rules

- Only the owning freelancer creates, updates or deletes a portfolio item.
- Only items whose deletedAt is null are listed, opened or edited.
- Deletion sets deletedAt; it does not hard-delete the item.
- The agreed database scope is title, description, technologies and project URL; media gallery, uploads and project role are excluded.

Normal Flow

1. The actor opens /profiles/[id]; /freelancer/profile and selects Update Portfolio.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.

3. The API checks authentication where required, authorization, resource existence and input validity.
4. Save validated edits on an owned, non-deleted portfolio item.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Save validated edits on an owned, non-deleted portfolio item.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful update portfolio produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.3.5 Function: Delete Portfolio (UC-09.05)

Function Trigger

The actor selects Delete Portfolio. Navigation: /profiles/[id]; /freelancer/profile.

Function Description

Actors / Roles

Freelancer.

Purpose

Set deletedAt and remove the item from active list and detail results.

Interface and Data Processing

`DELETE /profiles/portfolios/:id`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Profile, FreelancerProfile, PortfolioItem. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Delete Portfolio

Portfolio card grid with title, description and technologies; detail dialog; owner-only Add, Edit and Delete controls; confirmation before deletion.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Identifier	Positive resource ID from the route; ownership is checked by the API.

Validation and Business Rules

- Only the owning freelancer creates, updates or deletes a portfolio item.
- Only items whose deletedAt is null are listed, opened or edited.
- Deletion sets deletedAt; it does not hard-delete the item.
- The agreed database scope is title, description, technologies and project URL; media gallery, uploads and project role are excluded.

Normal Flow

1. The actor opens /profiles/[id]; /freelancer/profile and selects Delete Portfolio.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Set deletedAt and remove the item from active list and detail results.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.

Case	Description
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Set deletedAt and remove the item from active list and detail results.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful delete portfolio produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.4 Identity Verification

2.4.1 Function: Upload Identity Documents (UC-18.01)

Function Trigger

The actor selects Upload Identity Documents. Navigation: /account-profile (Identity Verification tab).

Function Description

Actors / Roles

Freelancer.

Purpose

Validate the active-request constraint, securely store the submitted evidence, and record the verification outcome for review.

Interface and Data Processing

`POST /identity-verifications/ai-check; POST /identity-verifications/documents`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses User, IdVerificationDocument, FreelancerProfile. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Upload Identity Documents

Document-type and front/back selectors; webcam liveness panel for AI submission; Submit; latest status badge, submission history and rejection instructions.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
documentType	Supported identity-document type.
front / back	Document images; both sides where required by document type.
selfie / liveness	Selfie and captured liveness frames for the AI path.
document file	Manual-path JPEG, PNG, WebP or PDF; maximum 5 MB under the identity business rule.

Validation and Business Rules

- Only an authenticated freelancer submits or reads their own verification status.
- At most one active PENDING or APPROVED request is allowed per freelancer.
- Documents are confidential; storage delivery and file endpoints must enforce authorization.
- AI success creates PENDING for administrator review; AI rejection records REJECTED with a reason. AI transport failure must not create a successful request.
- Rejected users may resubmit after correcting the stated problem. NOT_SUBMITTED is displayed when API status is null.

Normal Flow

1. The actor opens /account-profile (Identity Verification tab) and selects Upload Identity Documents.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Validate the active-request constraint, securely store the submitted evidence, and record the verification outcome for review.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
E5	An active PENDING/APPROVED request blocks resubmission. An unavailable AI service is reported without creating a successful verification record.

Output Data

Data	Definition
Result	Validate the active-request constraint, securely store the submitted evidence, and record the verification outcome for review.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful upload identity documents produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.4.2 Function: View Verification Status (UC-18.02)

Function Trigger

The actor selects View Verification Status. Navigation: /account-profile (Identity Verification tab).

Function Description

Actors / Roles

Freelancer.

Purpose

Return the latest verification state and submission history with rejection notes where present.

Interface and Data Processing

`GET /identity-verifications/status`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses User, IdVerificationDocument, FreelancerProfile. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

View Verification Status

Document-type and front/back selectors; webcam liveness panel for AI submission; Submit; latest status badge, submission history and rejection instructions.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Identifier	Authenticated user context; no other user ID can replace it.

Validation and Business Rules

- Only an authenticated freelancer submits or reads their own verification status.
- At most one active PENDING or APPROVED request is allowed per freelancer.
- Documents are confidential; storage delivery and file endpoints must enforce authorization.
- AI success creates PENDING for administrator review; AI rejection records REJECTED with a reason. AI transport failure must not create a successful request.
- Rejected users may resubmit after correcting the stated problem. NOT_SUBMITTED is displayed when API status is null.

Normal Flow

1. The actor opens /account-profile (Identity Verification tab) and selects View Verification Status.
2. The application submits the selected identifier or list filters.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Return the latest verification state and submission history with rejection notes where present.
5. The screen renders the response, or an explicit empty state for a valid empty collection.

Alternative Flow

Show an informative empty state for a valid empty result and allow retry after a transport failure.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
A1	A valid empty collection displays an empty state; it must not be treated as a missing account.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.
E5	An active PENDING/APPROVED request blocks resubmission. An unavailable AI service is reported without creating a successful verification record.

Output Data

Data	Definition
Result	Return the latest verification state and submission history with rejection notes where present.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful view verification status produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.5 Client Profile Management

2.5.1 Function: View Client Profile (UC-19.01)

Function Trigger

The actor selects View Client Profile. Navigation: /clients/[userId]; /client/profile.

Function Description

Actors / Roles

Guest, Client, Freelancer.

Purpose

Combine approved public Profile, ClientProfile and SocialLink fields without private user information.

Interface and Data Processing

`GET /clients/:userId`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses User, Profile, ClientProfile, SocialLink, ProfileRevision. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

View Client Profile

Public company header, description, website and social links; owner link to settings; Company tab with name, description and website fields.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Identifier	Positive resource ID from the route; ownership is checked by the API.

Validation and Business Rules

- Public client output contains approved company and profile fields, never password hashes, payment tokens or private identity documents.
- Only the authenticated Client updates their own company; the server derives the user ID from authentication.
- Apply the profile revision policy when strength is below 20 percent.
- Industry and location are outside the current ClientProfile schema. Hiring statistics are not part of the current response contract.

Normal Flow

1. The actor opens /clients/[userId]; /client/profile and selects View Client Profile.
2. The application submits the selected identifier or list filters.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Combine approved public Profile, ClientProfile and SocialLink fields without private user information.
5. The screen renders the response, or an explicit empty state for a valid empty collection.

Alternative Flow

Show an informative empty state for a valid empty result and allow retry after a transport failure.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
A1	A valid empty collection displays an empty state; it must not be treated as a missing account.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Combine approved public Profile, ClientProfile and SocialLink fields without private user information.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful view client profile produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.5.2 Function: Update Company Information (UC-19.02)

Function Trigger

The actor selects Update Company Information. Navigation: /clients/[userId]; /client/profile.

Function Description

Actors / Roles

Client.

Purpose

Save or submit the authenticated Client company information according to the revision policy.

Interface and Data Processing

`PUT /clients/me/profile`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses User, Profile, ClientProfile, SocialLink, ProfileRevision. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Update Company Information

Public company header, description, website and social links; owner link to settings; Company tab with name, description and website fields.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
companyName	Required trimmed company name, up to 255 characters.
companyDescription	Optional company description up to 5,000 characters.
website	Optional HTTP/HTTPS URL, at most 255 characters.

Validation and Business Rules

- Public client output contains approved company and profile fields, never password hashes, payment tokens or private identity documents.
- Only the authenticated Client updates their own company; the server derives the user ID from authentication.
- Apply the profile revision policy when strength is below 20 percent.
- Industry and location are outside the current ClientProfile schema. Hiring statistics are not part of the current response contract.

Normal Flow

1. The actor opens /clients/[userId]; /client/profile and selects Update Company Information.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Save or submit the authenticated Client company information according to the revision policy.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Save or submit the authenticated Client company information according to the revision policy.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful update company information produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.6 Social Links Management

2.6.1 Function: View Social Link List (UC-20.01)

Function Trigger

The actor selects View Social Link List. Navigation: /account-profile (Social Links); /client/profile; public profile.

Function Description

Actors / Roles

Client, Freelancer.

Purpose

Retrieve links for the current account settings or the public client response; public freelancer rendering is a tracked gap.

Interface and Data Processing

`GET /social-links; GET /clients/:userId`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Profile, SocialLink. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

View Social Link List

Platform selector and URL input; saved links with labels and external-link icons; delete confirmation; public link area with empty-state handling.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Selection	Current account or selected profile/contract; no pagination is assumed unless exposed by the endpoint.

Validation and Business Rules

- At most one link per platform per profile is allowed, including concurrent submissions.
- Only the owning user can add or hard-delete a social link.
- Open approved external links in a new tab with rel="noopener noreferrer".
- Profile.id links the SocialLink table; the authenticated User.id resolves that profile.

Normal Flow

1. The actor opens /account-profile (Social Links); /client/profile; public profile and selects View Social Link List.
2. The application submits the selected identifier or list filters.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Retrieve links for the current account settings or the public client response; public freelancer rendering is a tracked gap.
5. The screen renders the response, or an explicit empty state for a valid empty collection.

Alternative Flow

Show an informative empty state for a valid empty result and allow retry after a transport failure.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.

Case	Description
A1	A valid empty collection displays an empty state; it must not be treated as a missing account.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Retrieve links for the current account settings or the public client response; public freelancer rendering is a tracked gap.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful view social link list produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.6.2 Function: Add Social Link (UC-20.02)

Function Trigger

The actor selects Add Social Link. Navigation: /account-profile (Social Links); /client/profile; public profile.

Function Description

Actors / Roles

Client, Freelancer.

Purpose

Validate the platform and external URL, then insert one link on the authenticated user profile.

Interface and Data Processing

`POST /social-links`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Profile, SocialLink. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Add Social Link

Platform selector and URL input; saved links with labels and external-link icons; delete confirmation; public link area with empty-state handling.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
platform	One supported social platform enum value.
url	Required HTTP/HTTPS URL up to the SocialLink varchar(500) limit.
id	SocialLink.id of the link selected for deletion.

Validation and Business Rules

- At most one link per platform per profile is allowed, including concurrent submissions.
- Only the owning user can add or hard-delete a social link.
- Open approved external links in a new tab with rel="noopener noreferrer".
- Profile.id links the SocialLink table; the authenticated User.id resolves that profile.

Normal Flow

1. The actor opens /account-profile (Social Links); /client/profile; public profile and selects Add Social Link.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Validate the platform and external URL, then insert one link on the authenticated user profile.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Validate the platform and external URL, then insert one link on the authenticated user profile.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful add social link produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.6.3 Function: Delete Social Link (UC-20.03)

Function Trigger

The actor selects Delete Social Link. Navigation: /account-profile (Social Links); /client/profile; public profile.

Function Description

Actors / Roles

Client, Freelancer.

Purpose

Hard-delete the selected owned social link after confirmation.

Interface and Data Processing

`DELETE /social-links/:id`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Profile, SocialLink. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Delete Social Link

Platform selector and URL input; saved links with labels and external-link icons; delete confirmation; public link area with empty-state handling.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Identifier	Positive resource ID from the route; ownership is checked by the API.

Validation and Business Rules

- At most one link per platform per profile is allowed, including concurrent submissions.
- Only the owning user can add or hard-delete a social link.
- Open approved external links in a new tab with rel="noopener noreferrer".
- Profile.id links the SocialLink table; the authenticated User.id resolves that profile.

Normal Flow

1. The actor opens /account-profile (Social Links); /client/profile; public profile and selects Delete Social Link.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Hard-delete the selected owned social link after confirmation.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Hard-delete the selected owned social link after confirmation.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful delete social link produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.7 Favorite Freelancers

2.7.1 Function: View Favorite Freelancer List (UC-21.01)

Function Trigger

The actor selects View Favorite Freelancer List. Navigation: /client/profile (Favorites); /profiles/[id].

Function Description

Actors / Roles

Client.

Purpose

Load the current Client favorites and join public freelancer details.

Interface and Data Processing

`GET /favorites/freelancers`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses User, Profile, FreelancerProfile, FavoriteFreelancer. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

View Favorite Freelancer List

Favorite toggle on public freelancer profile; Favorites list with name, title, skills, profile link and Remove action; empty state.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Selection	Current account or selected profile/contract; no pagination is assumed unless exposed by the endpoint.

Validation and Business Rules

- Only an authenticated Client manages their own favorites.
- Reject self-favoriting and targets without an active Freelancer role.
- The composite key (clientId, freelancerId) prevents duplicate relationships.
- Removal hard-deletes only the relationship; no freelancer account or profile is deleted.
- A successful add must be visible after list reload, including a new freelancer with minimal professional data.

Normal Flow

1. The actor opens /client/profile (Favorites); /profiles/[id] and selects View Favorite Freelancer List.
2. The application submits the selected identifier or list filters.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Load the current Client favorites and join public freelancer details.
5. The screen renders the response, or an explicit empty state for a valid empty collection.

Alternative Flow

Show an informative empty state for a valid empty result and allow retry after a transport failure.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
A1	A valid empty collection displays an empty state; it must not be treated as a missing account.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Load the current Client favorites and join public freelancer details.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful view favorite freelancer list produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.7.2 Function: Add Freelancer to Favorites (UC-21.02)

Function Trigger

The actor selects Add Freelancer to Favorites. Navigation: /client/profile (Favorites); /profiles/[id].

Function Description

Actors / Roles

Client.

Purpose

Insert a unique relationship from the current Client to an active freelancer.

Interface and Data Processing

`POST /favorites/freelancers/:freelancerId`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses User, Profile, FreelancerProfile, FavoriteFreelancer. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Add Freelancer to Favorites

Favorite toggle on public freelancer profile; Favorites list with name, title, skills, profile link and Remove action; empty state.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
freelancerId	Target freelancer User.id, not Profile.id.

Validation and Business Rules

- Only an authenticated Client manages their own favorites.
- Reject self-favoriting and targets without an active Freelancer role.
- The composite key (clientId, freelancerId) prevents duplicate relationships.
- Removal hard-deletes only the relationship; no freelancer account or profile is deleted.
- A successful add must be visible after list reload, including a new freelancer with minimal professional data.

Normal Flow

1. The actor opens /client/profile (Favorites); /profiles/[id] and selects Add Freelancer to Favorites.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Insert a unique relationship from the current Client to an active freelancer.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Insert a unique relationship from the current Client to an active freelancer.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful add freelancer to favorites produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.7.3 Function: Remove Freelancer from Favorites (UC-21.03)

Function Trigger

The actor selects Remove Freelancer from Favorites. Navigation: /client/profile (Favorites); /profiles/[id].

Function Description

Actors / Roles

Client.

Purpose

Hard-delete the current Client relationship and update the favorite toggle and list.

Interface and Data Processing

`DELETE /favorites/freelancers/:freelancerId`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses User, Profile, FreelancerProfile, FavoriteFreelancer. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Remove Freelancer from Favorites

Favorite toggle on public freelancer profile; Favorites list with name, title, skills, profile link and Remove action; empty state.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Identifier	Positive resource ID from the route; ownership is checked by the API.

Validation and Business Rules

- Only an authenticated Client manages their own favorites.
- Reject self-favoriting and targets without an active Freelancer role.
- The composite key (clientId, freelancerId) prevents duplicate relationships.
- Removal hard-deletes only the relationship; no freelancer account or profile is deleted.
- A successful add must be visible after list reload, including a new freelancer with minimal professional data.

Normal Flow

1. The actor opens /client/profile (Favorites); /profiles/[id] and selects Remove Freelancer from Favorites.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Hard-delete the current Client relationship and update the favorite toggle and list.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Hard-delete the current Client relationship and update the favorite toggle and list.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful remove freelancer from favorites produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.8 Following Management

2.8.1 Function: View Following List (UC-29.01)

Function Trigger

The actor selects View Following List. Navigation: /client/profile (Following); /profiles/[id].

Function Description

Actors / Roles

Client.

Purpose

Load freelancer relationships owned by the current Client and show the following state.

Interface and Data Processing

`GET /following/freelancers`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses User, FollowFreelancer, Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

View Following List

Following cards with profile links and Unfollow; discovery section and Follow action; empty-state message.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Selection	Current account or selected profile/contract; no pagination is assumed unless exposed by the endpoint.

Validation and Business Rules

- Only a Client follows or unfollows an active freelancer; self-follow is rejected.
- A unique (clientId, freelancerId) mapping prevents duplicates.
- Create the follow and its recipient notification together in a transaction.
- Unfollow hard-deletes the mapping without deleting either user.

Normal Flow

1. The actor opens /client/profile (Following); /profiles/[id] and selects View Following List.
2. The application submits the selected identifier or list filters.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Load freelancer relationships owned by the current Client and show the following state.
5. The screen renders the response, or an explicit empty state for a valid empty collection.

Alternative Flow

Show an informative empty state for a valid empty result and allow retry after a transport failure.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.

Case	Description
A1	A valid empty collection displays an empty state; it must not be treated as a missing account.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Load freelancer relationships owned by the current Client and show the following state.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful view following list produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.8.2 Function: Follow Freelancer (UC-29.02)

Function Trigger

The actor selects Follow Freelancer. Navigation: /client/profile (Following); /profiles/[id].

Function Description

Actors / Roles

Client.

Purpose

Create a unique follow mapping and notify the target freelancer.

Interface and Data Processing

`POST /following/freelancers/:freelancerId`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses User, FollowFreelancer, Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Follow Freelancer

Following cards with profile links and Unfollow; discovery section and Follow action; empty-state message.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
freelancerId	Target freelancer User.id.

Validation and Business Rules

- Only a Client follows or unfollows an active freelancer; self-follow is rejected.
- A unique (clientId, freelancerId) mapping prevents duplicates.
- Create the follow and its recipient notification together in a transaction.
- Unfollow hard-deletes the mapping without deleting either user.

Normal Flow

1. The actor opens /client/profile (Following); /profiles/[id] and selects Follow Freelancer.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.

4. Create a unique follow mapping and notify the target freelancer.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Create a unique follow mapping and notify the target freelancer.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful follow freelancer produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.8.3 Function: Unfollow Freelancer (UC-29.03)

Function Trigger

The actor selects Unfollow Freelancer. Navigation: /client/profile (Following); /profiles/[id].

Function Description

Actors / Roles

Client.

Purpose

Remove only the current Client follow relationship.

Interface and Data Processing

`DELETE /following/freelancers/:freelancerId`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses User, FollowFreelancer, Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Unfollow Freelancer

Following cards with profile links and Unfollow; discovery section and Follow action; empty-state message.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Identifier	Positive resource ID from the route; ownership is checked by the API.

Validation and Business Rules

- Only a Client follows or unfollows an active freelancer; self-follow is rejected.
- A unique (clientId, freelancerId) mapping prevents duplicates.
- Create the follow and its recipient notification together in a transaction.
- Unfollow hard-deletes the mapping without deleting either user.

Normal Flow

1. The actor opens /client/profile (Following); /profiles/[id] and selects Unfollow Freelancer.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Remove only the current Client follow relationship.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.

Case	Description
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Remove only the current Client follow relationship.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful unfollow freelancer produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.9 Notification Management

2.9.1 Function: View Notification List (UC-30.01)

Function Trigger

The actor selects View Notification List. Navigation: Header notification menu; /notifications.

Function Description

Actors / Roles

Client, Freelancer, Expert, Admin.

Purpose

Load the recipient active notifications in reverse chronological order.

Interface and Data Processing

`GET /notifications`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

View Notification List

Notification list with title, content, timestamp and unread indication; Mark as Read, Mark All as Read and Delete controls.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Selection	Current account or selected profile/contract; no pagination is assumed unless exposed by the endpoint.

Validation and Business Rules

- A user reads and changes only their own notifications.
- Exclude rows with deletedAt set and sort by createdAt descending.
- Mark-read sets isRead=true; mark-all affects only the recipient unread active rows.
- Deletion is a soft delete and updates the displayed list and unread count.

Normal Flow

1. The actor opens Header notification menu; /notifications and selects View Notification List.
2. The application submits the selected identifier or list filters.
3. The API checks authentication where required, authorization, resource existence and input validity.

4. Load the recipient active notifications in reverse chronological order.
5. The screen renders the response, or an explicit empty state for a valid empty collection.

Alternative Flow

Show an informative empty state for a valid empty result and allow retry after a transport failure.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
A1	A valid empty collection displays an empty state; it must not be treated as a missing account.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Load the recipient active notifications in reverse chronological order.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful view notification list produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.9.2 Function: Mark Notification as Read (UC-30.02)

Function Trigger

The actor selects Mark Notification as Read. Navigation: Header notification menu; /notifications.

Function Description

Actors / Roles

Client, Freelancer, Expert, Admin.

Purpose

Set isRead to true on one owned notification.

Interface and Data Processing

`PATCH /notifications/:id/read`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Mark Notification as Read

Notification list with title, content, timestamp and unread indication; Mark as Read, Mark All as Read and Delete controls.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Accepted input	notification id

Validation and Business Rules

- A user reads and changes only their own notifications.
- Exclude rows with deletedAt set and sort by createdAt descending.
- Mark-read sets isRead=true; mark-all affects only the recipient unread active rows.
- Deletion is a soft delete and updates the displayed list and unread count.

Normal Flow

1. The actor opens Header notification menu; /notifications and selects Mark Notification as Read.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Set isRead to true on one owned notification.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.

Case	Description
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Set isRead to true on one owned notification.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful mark notification as read produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.9.3 Function: Mark All Notifications as Read (UC-30.03)

Function Trigger

The actor selects Mark All Notifications as Read. Navigation: Header notification menu; /notifications.

Function Description

Actors / Roles

Client, Freelancer, Expert, Admin.

Purpose

Update the current recipient unread, non-deleted notifications together.

Interface and Data Processing

`PATCH /notifications/read-all`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Mark All Notifications as Read

Notification list with title, content, timestamp and unread indication; Mark as Read, Mark All as Read and Delete controls.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Accepted input	authenticated recipient; no notification id

Validation and Business Rules

- A user reads and changes only their own notifications.
- Exclude rows with deletedAt set and sort by createdAt descending.
- Mark-read sets isRead=true; mark-all affects only the recipient unread active rows.
- Deletion is a soft delete and updates the displayed list and unread count.

Normal Flow

1. The actor opens Header notification menu; /notifications and selects Mark All Notifications as Read.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Update the current recipient unread, non-deleted notifications together.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Update the current recipient unread, non-deleted notifications together.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful mark all notifications as read produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.9.4 Function: Delete Notification (UC-30.04)

Function Trigger

The actor selects Delete Notification. Navigation: Header notification menu; /notifications.

Function Description

Actors / Roles

Client, Freelancer, Expert, Admin.

Purpose

Set deletedAt on the owned notification and hide it from the list.

Interface and Data Processing

`DELETE /notifications/:id`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Delete Notification

Notification list with title, content, timestamp and unread indication; Mark as Read, Mark All as Read and Delete controls.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Identifier	Positive resource ID from the route; ownership is checked by the API.

Validation and Business Rules

- A user reads and changes only their own notifications.
- Exclude rows with deletedAt set and sort by createdAt descending.
- Mark-read sets isRead=true; mark-all affects only the recipient unread active rows.
- Deletion is a soft delete and updates the displayed list and unread count.

Normal Flow

1. The actor opens Header notification menu; /notifications and selects Delete Notification.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Set deletedAt on the owned notification and hide it from the list.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.

Case	Description
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Set deletedAt on the owned notification and hide it from the list.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful delete notification produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.10 Role Switching and Enrollment

2.10.1 Function: Switch Role (UC-31.01)

Function Trigger

The actor selects Switch Role. Navigation: Header role menu; role enrollment dialog.

Function Description

Actors / Roles

Client, Freelancer.

Purpose

Change the primary role to an already assigned role and issue updated authentication tokens.

Interface and Data Processing

`POST /auth/switch-role`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses User, Role, UserRole, Session. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Switch Role

Header showing current role, assigned role options and Join Role action; enrollment confirmation; destination dashboard after success.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
role	JSON field role; allowed values are Client or Freelancer. Expert and Admin are not self-enrollment options.

Validation and Business Rules

- Switch only to a role already assigned to the authenticated user.
- Self-enrollment permits only Client or Freelancer; reject unsupported roles and duplicate UserRole mappings.
- Maintain exactly one active primary role for an account; role-context changes issue updated session tokens.
- Switching roles does not create a second user account or grant access to other users records.

Normal Flow

1. The actor opens Header role menu; role enrollment dialog and selects Switch Role.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Change the primary role to an already assigned role and issue updated authentication tokens.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Change the primary role to an already assigned role and issue updated authentication tokens.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful switch role produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.10.2 Function: Join New Role (UC-31.02)

Function Trigger

The actor selects Join New Role. Navigation: Header role menu; role enrollment dialog.

Function Description

Actors / Roles

Client, Freelancer.

Purpose

Enroll in an allowed new role without duplicating the account or granting administrator privileges.

Interface and Data Processing

`POST /auth/join-role`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses User, Role, UserRole, Session. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Join New Role

Header showing current role, assigned role options and Join Role action; enrollment confirmation; destination dashboard after success.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
role	JSON field role; allowed values are Client or Freelancer. Expert and Admin are not self-enrollment options.

Validation and Business Rules

- Switch only to a role already assigned to the authenticated user.
- Self-enrollment permits only Client or Freelancer; reject unsupported roles and duplicate UserRole mappings.
- Maintain exactly one active primary role for an account; role-context changes issue updated session tokens.
- Switching roles does not create a second user account or grant access to other users records.

Normal Flow

1. The actor opens Header role menu; role enrollment dialog and selects Join New Role.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Enroll in an allowed new role without duplicating the account or granting administrator privileges.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Enroll in an allowed new role without duplicating the account or granting administrator privileges.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful join new role produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.11 Contract Review Management

2.11.1 Function: View Review List (UC-39.01)

Function Trigger

The actor selects View Review List. Navigation: /client/contracts/[slug]; /freelancer/contracts/[slug].

Function Description

Actors / Roles

Client, Freelancer.

Purpose

Retrieve reviews and active responses for a contract after verifying participant access.

Interface and Data Processing

`GET /reviews/contracts/:contractId`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Contract, Review, Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

View Review List

Contract Reviews section with author, rating, breakdown and comment; Write Review dialog; author-only Edit and Delete actions.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Selection	Current account or selected profile/contract; no pagination is assumed unless exposed by the endpoint.

Validation and Business Rules

- Only contract participants can list or view contract reviews.
- A review can be created only after the contract is COMPLETED, for the other party.
- The pair (contractId, reviewerId) is unique.
- Only the review author edits or soft-deletes their review.
- An update must contain at least one accepted field. The current schema does not require a non-empty comment or an integer overall rating.

Normal Flow

1. The actor opens /client/contracts/[slug]; /freelancer/contracts/[slug] and selects View Review List.
2. The application submits the selected identifier or list filters.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Retrieve reviews and active responses for a contract after verifying participant access.
5. The screen renders the response, or an explicit empty state for a valid empty collection.

Alternative Flow

Show an informative empty state for a valid empty result and allow retry after a transport failure.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
A1	A valid empty collection displays an empty state; it must not be treated as a missing account.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Retrieve reviews and active responses for a contract after verifying participant access.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful view review list produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.11.2 Function: View Review Detail (UC-39.02)

Function Trigger

The actor selects View Review Detail. Navigation: /client/contracts/[slug]; /freelancer/contracts/[slug].

Function Description

Actors / Roles

Client, Freelancer.

Purpose

Retrieve one review and response after checking contract participation.

Interface and Data Processing

`GET /reviews/:reviewId`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Contract, Review, Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

View Review Detail

Contract Reviews section with author, rating, breakdown and comment; Write Review dialog; author-only Edit and Delete actions.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Identifier	Positive resource ID from the route; ownership is checked by the API.

Validation and Business Rules

- Only contract participants can list or view contract reviews.
- A review can be created only after the contract is COMPLETED, for the other party.
- The pair (contractId, reviewerId) is unique.
- Only the review author edits or soft-deletes their review.
- An update must contain at least one accepted field. The current schema does not require a non-empty comment or an integer overall rating.

Normal Flow

1. The actor opens /client/contracts/[slug]; /freelancer/contracts/[slug] and selects View Review Detail.
2. The application submits the selected identifier or list filters.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Retrieve one review and response after checking contract participation.
5. The screen renders the response, or an explicit empty state for a valid empty collection.

Alternative Flow

Show an informative empty state for a valid empty result and allow retry after a transport failure.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
A1	A valid empty collection displays an empty state; it must not be treated as a missing account.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Retrieve one review and response after checking contract participation.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful view review detail produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.11.3 Function: Write Review (UC-39.03)

Function Trigger

The actor selects Write Review. Navigation: /client/contracts/[slug]; /freelancer/contracts/[slug].

Function Description

Actors / Roles

Client, Freelancer.

Purpose

Create one review of the other participant in a completed contract and notify that participant.

Interface and Data Processing

`POST /reviews/contracts/:contractId`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Contract, Review, Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Write Review

Contract Reviews section with author, rating, breakdown and comment; Write Review dialog; author-only Edit and Delete actions.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
overallRating	Required numeric rating from 1 to 5 when creating.
breakdown	Optional ratings map: at most 10 criteria, names up to 50 characters, values 1–5.
comment	Optional trimmed feedback, up to 3,000 characters.
contractId / reviewId	Positive database identifiers; contract page URLs use slug.

Validation and Business Rules

- Only contract participants can list or view contract reviews.
- A review can be created only after the contract is COMPLETED, for the other party.
- The pair (contractId, reviewerId) is unique.
- Only the review author edits or soft-deletes their review.
- An update must contain at least one accepted field. The current schema does not require a non-empty comment or an integer overall rating.

Normal Flow

1. The actor opens /client/contracts/[slug]; /freelancer/contracts/[slug] and selects Write Review.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.

4. Create one review of the other participant in a completed contract and notify that participant.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Create one review of the other participant in a completed contract and notify that participant.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful write review produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.11.4 Function: Update Review (UC-39.04)

Function Trigger

The actor selects Update Review. Navigation: /client/contracts/[slug]; /freelancer/contracts/[slug].

Function Description

Actors / Roles

Client, Freelancer.

Purpose

Update accepted fields of an existing review authored by the current user.

Interface and Data Processing

`PATCH /reviews/:reviewId`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Contract, Review, Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Update Review

Contract Reviews section with author, rating, breakdown and comment; Write Review dialog; author-only Edit and Delete actions.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
overallRating	Required numeric rating from 1 to 5 when creating.
breakdown	Optional ratings map: at most 10 criteria, names up to 50 characters, values 1–5.
comment	Optional trimmed feedback, up to 3,000 characters.
contractId / reviewId	Positive database identifiers; contract page URLs use slug.

Validation and Business Rules

- Only contract participants can list or view contract reviews.
- A review can be created only after the contract is COMPLETED, for the other party.
- The pair (contractId, reviewerId) is unique.
- Only the review author edits or soft-deletes their review.
- An update must contain at least one accepted field. The current schema does not require a non-empty comment or an integer overall rating.

Normal Flow

1. The actor opens /client/contracts/[slug]; /freelancer/contracts/[slug] and selects Update Review.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Update accepted fields of an existing review authored by the current user.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Update accepted fields of an existing review authored by the current user.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful update review produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.11.5 Function: Delete Review (UC-39.05)

Function Trigger

The actor selects Delete Review. Navigation: /client/contracts/[slug]; /freelancer/contracts/[slug].

Function Description

Actors / Roles

Client, Freelancer.

Purpose

Soft-delete the owned review so it no longer appears as active feedback.

Interface and Data Processing

`DELETE /reviews/:reviewId`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Contract, Review, Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Delete Review

Contract Reviews section with author, rating, breakdown and comment; Write Review dialog; author-only Edit and Delete actions.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Identifier	Positive resource ID from the route; ownership is checked by the API.

Validation and Business Rules

- Only contract participants can list or view contract reviews.
- A review can be created only after the contract is COMPLETED, for the other party.
- The pair (contractId, reviewerId) is unique.
- Only the review author edits or soft-deletes their review.
- An update must contain at least one accepted field. The current schema does not require a non-empty comment or an integer overall rating.

Normal Flow

1. The actor opens /client/contracts/[slug]; /freelancer/contracts/[slug] and selects Delete Review.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Soft-delete the owned review so it no longer appears as active feedback.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Soft-delete the owned review so it no longer appears as active feedback.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful delete review produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.12 Review Response Management

2.12.1 Function: Respond to Review (UC-40.01)

Function Trigger

The actor selects Respond to Review. Navigation: Contract Reviews section.

Function Description

Actors / Roles

Client, Freelancer.

Purpose

Create an active response from the reviewee and notify the review author.

Interface and Data Processing

`POST /reviews/:reviewId/response`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Review, ReviewResponse, Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Respond to Review

Response beneath its review with author and time; Respond/Edit form; response-author-only Delete confirmation.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
responseText	Required trimmed response, 1–3,000 characters.
reviewId / responseId	Review being answered or existing response identifier.

Validation and Business Rules

- Only the user being reviewed may respond.
- There is at most one active response per review.
- Only the response author edits or soft-deletes it.
- Soft-deleted responses must not be rendered as active feedback.

Normal Flow

1. The actor opens Contract Reviews section and selects Respond to Review.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Create an active response from the reviewee and notify the review author.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.

Case	Description
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Create an active response from the reviewee and notify the review author.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful respond to review produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.12.2 Function: Update Review Response (UC-40.02)

Function Trigger

The actor selects Update Review Response. Navigation: Contract Reviews section.

Function Description

Actors / Roles

Client, Freelancer.

Purpose

Save validated text in the current user response.

Interface and Data Processing

`PATCH /reviews/responses/:responseId`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Review, ReviewResponse, Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Update Review Response

Response beneath its review with author and time; Respond/Edit form; response-author-only Delete confirmation.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
responseText	Required trimmed response, 1–3,000 characters.
reviewId / responseId	Review being answered or existing response identifier.

Validation and Business Rules

- Only the user being reviewed may respond.
- There is at most one active response per review.
- Only the response author edits or soft-deletes it.
- Soft-deleted responses must not be rendered as active feedback.

Normal Flow

1. The actor opens Contract Reviews section and selects Update Review Response.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.

3. The API checks authentication where required, authorization, resource existence and input validity.
4. Save validated text in the current user response.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Save validated text in the current user response.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful update review response produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.12.3 Function: Delete Review Response (UC-40.03)

Function Trigger

The actor selects Delete Review Response. Navigation: Contract Reviews section.

Function Description

Actors / Roles

Client, Freelancer.

Purpose

Soft-delete the response authored by the current user.

Interface and Data Processing

`DELETE /reviews/responses/:responseId`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Review, ReviewResponse, Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Delete Review Response

Response beneath its review with author and time; Respond/Edit form; response-author-only Delete confirmation.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Identifier	Positive resource ID from the route; ownership is checked by the API.

Validation and Business Rules

- Only the user being reviewed may respond.
- There is at most one active response per review.
- Only the response author edits or soft-deletes it.
- Soft-deleted responses must not be rendered as active feedback.

Normal Flow

1. The actor opens Contract Reviews section and selects Delete Review Response.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Soft-delete the response authored by the current user.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.

Case	Description
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Soft-delete the response authored by the current user.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful delete review response produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.13 General Account Profile

2.13.1 Function: View Profile (UC-41.01)

Function Trigger

The actor selects View Profile. Navigation: /account-profile (General Settings).

Function Description

Actors / Roles

Client, Freelancer, Expert, Admin.

Purpose

Read the authenticated account profile, email and assigned roles.

Interface and Data Processing

`GET /account-profile`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses User, Profile, ProfileRevision. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

View Profile

Read-only email and role summary; display name and bio form; avatar picker; separate password-change form and confirmation field.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Identifier	Authenticated user context; no other user ID can replace it.

Validation and Business Rules

- The server resolves the account from the session; users cannot edit another account via body IDs.
- Verify the current password before hashing and storing a replacement; do not log credentials.
- Apply the profile revision policy to general profile changes.
- Avatar files must be validated; missing stored files return 404 without terminating the API process.

Normal Flow

1. The actor opens /account-profile (General Settings) and selects View Profile.
2. The application submits the selected identifier or list filters.

3. The API checks authentication where required, authorization, resource existence and input validity.
4. Read the authenticated account profile, email and assigned roles.
5. The screen renders the response, or an explicit empty state for a valid empty collection.

Alternative Flow

Show an informative empty state for a valid empty result and allow retry after a transport failure.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
A1	A valid empty collection displays an empty state; it must not be treated as a missing account.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Read the authenticated account profile, email and assigned roles.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful view profile produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.13.2 Function: Update Profile (UC-41.02)

Function Trigger

The actor selects Update Profile. Navigation: /account-profile (General Settings).

Function Description

Actors / Roles

Client, Freelancer, Expert, Admin.

Purpose

Save or submit general display name and biography according to profile strength.

Interface and Data Processing

`PUT /account-profile`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses User, Profile, ProfileRevision. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Update Profile

Read-only email and role summary; display name and bio form; avatar picker; separate password-change form and confirmation field.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Accepted input	displayName; optional bio

Validation and Business Rules

- The server resolves the account from the session; users cannot edit another account via body IDs.
- Verify the current password before hashing and storing a replacement; do not log credentials.
- Apply the profile revision policy to general profile changes.
- Avatar files must be validated; missing stored files return 404 without terminating the API process.

Normal Flow

1. The actor opens /account-profile (General Settings) and selects Update Profile.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Save or submit general display name and biography according to profile strength.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Save or submit general display name and biography according to profile strength.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful update profile produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.13.3 Function: Change Password (UC-41.03)

Function Trigger

The actor selects Change Password. Navigation: /account-profile (General Settings).

Function Description

Actors / Roles

Client, Freelancer, Expert, Admin.

Purpose

Verify the current credential, hash the confirmed replacement and update the password.

Interface and Data Processing

`PUT /account-profile/password`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses User, Profile, ProfileRevision. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Change Password
Read-only email and role summary; display name and bio form; avatar picker; separate password-change form and confirmation field.
Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Accepted input	currentPassword; newPassword; confirmPassword

Validation and Business Rules

- The server resolves the account from the session; users cannot edit another account via body IDs.
- Verify the current password before hashing and storing a replacement; do not log credentials.
- Apply the profile revision policy to general profile changes.
- Avatar files must be validated; missing stored files return 404 without terminating the API process.

Normal Flow

1. The actor opens /account-profile (General Settings) and selects Change Password.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Verify the current credential, hash the confirmed replacement and update the password.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Verify the current credential, hash the confirmed replacement and update the password.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful change password produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.13.4 Function: Upload Avatar (UC-41.04)

Function Trigger

The actor selects Upload Avatar. Navigation: /account-profile (General Settings).

Function Description

Actors / Roles

Client, Freelancer, Expert, Admin.

Purpose

Validate image content and size, store it, and update the owned avatar URL.

Interface and Data Processing

`POST /account-profile/avatar`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses User, Profile, ProfileRevision. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Upload Avatar

Read-only email and role summary; display name and bio form; avatar picker; separate password-change form and confirmation field.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Accepted input	multipart file

Validation and Business Rules

- The server resolves the account from the session; users cannot edit another account via body IDs.
- Verify the current password before hashing and storing a replacement; do not log credentials.
- Apply the profile revision policy to general profile changes.
- Avatar files must be validated; missing stored files return 404 without terminating the API process.

Normal Flow

1. The actor opens /account-profile (General Settings) and selects Upload Avatar.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Validate image content and size, store it, and update the owned avatar URL.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Validate image content and size, store it, and update the owned avatar URL.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful upload avatar produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.14 Profile Revision Review and Expert Profile

2.14.1 Function: View Pending Profile Update Request List (UC-47.01)

Function Trigger

The actor selects View Pending Profile Update Request List. Navigation: /admin/profile-revisions; /expert/profile.

Function Description

Actors / Roles

Admin.

Purpose

Filter and paginate revisions, defaulting the review workflow to pending requests.

Interface and Data Processing

`GET /admin/profile-revisions`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses ProfileRevision, Profile, FreelancerProfile, ClientProfile, Expert, Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

View Pending Profile Update Request List

Admin filter bar and revision table; current/proposed comparison panel; Approve/Reject controls and note field; Expert form with submission status.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Selection	Validated page, limit, search, status and profileType filters.

Validation and Business Rules

- Only an authorized administrator can inspect or decide profile revisions.
- Only PENDING revisions may be approved or rejected. Decision and profile writes are transactional.
- Approval applies validated proposed data to the correct subtype; rejection retains existing public data.
- Record reviewer, decision time and notes; preserve history and notify the requester.
- An Expert submits their own update as a pending revision. Administrative expert editing is a separate authorized operation.

Normal Flow

1. The actor opens /admin/profile-revisions; /expert/profile and selects View Pending Profile Update Request List.
2. The application submits the selected identifier or list filters.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Filter and paginate revisions, defaulting the review workflow to pending requests.
5. The screen renders the response, or an explicit empty state for a valid empty collection.

Alternative Flow

Show an informative empty state for a valid empty result and allow retry after a transport failure.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
A1	A valid empty collection displays an empty state; it must not be treated as a missing account.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Filter and paginate revisions, defaulting the review workflow to pending requests.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful view pending profile update request list produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.14.2 Function: View Profile Update Request Detail (UC-47.02)

Function Trigger

The actor selects View Profile Update Request Detail. Navigation: /admin/profile-revisions; /expert/profile.

Function Description

Actors / Roles

Admin.

Purpose

Display the current and proposed snapshots, requester, profile type and prior review notes.

Interface and Data Processing

`GET /admin/profile-revisions/:id`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses ProfileRevision, Profile, FreelancerProfile, ClientProfile, Expert, Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

View Profile Update Request Detail

Admin filter bar and revision table; current/proposed comparison panel; Approve/Reject controls and note field; Expert form with submission status.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Identifier	Positive resource ID from the route; ownership is checked by the API.

Validation and Business Rules

- Only an authorized administrator can inspect or decide profile revisions.
- Only PENDING revisions may be approved or rejected. Decision and profile writes are transactional.
- Approval applies validated proposed data to the correct subtype; rejection retains existing public data.
- Record reviewer, decision time and notes; preserve history and notify the requester.
- An Expert submits their own update as a pending revision. Administrative expert editing is a separate authorized operation.

Normal Flow

1. The actor opens /admin/profile-revisions; /expert/profile and selects View Profile Update Request Detail.
2. The application submits the selected identifier or list filters.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Display the current and proposed snapshots, requester, profile type and prior review notes.
5. The screen renders the response, or an explicit empty state for a valid empty collection.

Alternative Flow

Show an informative empty state for a valid empty result and allow retry after a transport failure.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
A1	A valid empty collection displays an empty state; it must not be treated as a missing account.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Display the current and proposed snapshots, requester, profile type and prior review notes.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful view profile update request detail produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.14.3 Function: Approve Profile Update Request (UC-47.03)

Function Trigger

The actor selects Approve Profile Update Request. Navigation: /admin/profile-revisions; /expert/profile.

Function Description

Actors / Roles

Admin.

Purpose

Apply validated proposed changes and record an APPROVED decision atomically.

Interface and Data Processing

`PATCH /admin/profile-revisions/:id/approve`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses ProfileRevision, Profile, FreelancerProfile, ClientProfile, Expert, Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Approve Profile Update Request

Admin filter bar and revision table; current/proposed comparison panel; Approve/Reject controls and note field; Expert form with submission status.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Accepted input	revision id; optional reviewNotes

Validation and Business Rules

- Only an authorized administrator can inspect or decide profile revisions.
- Only PENDING revisions may be approved or rejected. Decision and profile writes are transactional.
- Approval applies validated proposed data to the correct subtype; rejection retains existing public data.
- Record reviewer, decision time and notes; preserve history and notify the requester.
- An Expert submits their own update as a pending revision. Administrative expert editing is a separate authorized operation.

Normal Flow

1. The actor opens /admin/profile-revisions; /expert/profile and selects Approve Profile Update Request.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Apply validated proposed changes and record an APPROVED decision atomically.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Apply validated proposed changes and record an APPROVED decision atomically.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful approve profile update request produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.14.4 Function: Reject Profile Update Request (UC-47.04)

Function Trigger

The actor selects Reject Profile Update Request. Navigation: /admin/profile-revisions; /expert/profile.

Function Description

Actors / Roles

Admin.

Purpose

Keep approved profile data unchanged and record REJECTED with an explanatory note.

Interface and Data Processing

`PATCH /admin/profile-revisions/:id/reject`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses ProfileRevision, Profile, FreelancerProfile, ClientProfile, Expert, Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Reject Profile Update Request

Admin filter bar and revision table; current/proposed comparison panel; Approve/Reject controls and note field; Expert form with submission status.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Accepted input	revision id; required reviewNotes

Validation and Business Rules

- Only an authorized administrator can inspect or decide profile revisions.
- Only PENDING revisions may be approved or rejected. Decision and profile writes are transactional.
- Approval applies validated proposed data to the correct subtype; rejection retains existing public data.
- Record reviewer, decision time and notes; preserve history and notify the requester.
- An Expert submits their own update as a pending revision. Administrative expert editing is a separate authorized operation.

Normal Flow

1. The actor opens /admin/profile-revisions; /expert/profile and selects Reject Profile Update Request.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Keep approved profile data unchanged and record REJECTED with an explanatory note.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Keep approved profile data unchanged and record REJECTED with an explanatory note.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful reject profile update request produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.14.5 Function: View Profile Update Request History (UC-47.05)

Function Trigger

The actor selects View Profile Update Request History. Navigation: /admin/profile-revisions; /expert/profile.

Function Description

Actors / Roles

Admin.

Purpose

Allow Admin to filter historical decisions; a requester can read only their latest revision for a profile type.

Interface and Data Processing

`GET /admin/profile-revisions; GET /profile-revisions/me?type=....` The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses ProfileRevision, Profile, FreelancerProfile, ClientProfile, Expert, Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

View Profile Update Request History

Admin filter bar and revision table; current/proposed comparison panel; Approve/Reject controls and note field; Expert form with submission status.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Selection	Validated page, limit, search, status and profileType filters.

Validation and Business Rules

- Only an authorized administrator can inspect or decide profile revisions.
- Only PENDING revisions may be approved or rejected. Decision and profile writes are transactional.
- Approval applies validated proposed data to the correct subtype; rejection retains existing public data.
- Record reviewer, decision time and notes; preserve history and notify the requester.
- An Expert submits their own update as a pending revision. Administrative expert editing is a separate authorized operation.

Normal Flow

1. The actor opens /admin/profile-revisions; /expert/profile and selects View Profile Update Request History.
2. The application submits the selected identifier or list filters.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Allow Admin to filter historical decisions; a requester can read only their latest revision for a profile type.
5. The screen renders the response, or an explicit empty state for a valid empty collection.

Alternative Flow

Show an informative empty state for a valid empty result and allow retry after a transport failure.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
A1	A valid empty collection displays an empty state; it must not be treated as a missing account.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Allow Admin to filter historical decisions; a requester can read only their latest revision for a profile type.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful view profile update request history produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.14.6 Function: Update Profile Expert (UC-47.06)

Function Trigger

The actor selects Update Profile Expert. Navigation: /admin/profile-revisions; /expert/profile.

Function Description

Actors / Roles

Expert.

Purpose

Submit the owned Expert profile as a pending revision for administrator review.

Interface and Data Processing

`PUT /expert-profile/me`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses ProfileRevision, Profile, FreelancerProfile, ClientProfile, Expert, Notification. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Update Profile Expert

Admin filter bar and revision table; current/proposed comparison panel; Approve/Reject controls and note field; Expert form with submission status.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Accepted input	expert profile fields

Validation and Business Rules

- Only an authorized administrator can inspect or decide profile revisions.
- Only PENDING revisions may be approved or rejected. Decision and profile writes are transactional.
- Approval applies validated proposed data to the correct subtype; rejection retains existing public data.
- Record reviewer, decision time and notes; preserve history and notify the requester.
- An Expert submits their own update as a pending revision. Administrative expert editing is a separate authorized operation.

Normal Flow

1. The actor opens /admin/profile-revisions; /expert/profile and selects Update Profile Expert.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Submit the owned Expert profile as a pending revision for administrator review.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Submit the owned Expert profile as a pending revision for administrator review.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful update profile expert produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.15 Freelancer CV Management

2.15.1 Function: Upload CV (UC-07.03)

Function Trigger

The actor selects Upload CV. Navigation: /freelancer/profile; /profiles/[id].

Function Description

Actors / Roles

Freelancer.

Purpose

Validate a PDF up to 10 MB and attach it to the owned FreelancerProfile.

Interface and Data Processing

`POST /profiles/:id/cv`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Profile, FreelancerProfile, ProfileRevision. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Upload CV

Current CV file name; Upload or Replace PDF picker; View/Download and Delete confirmation; progress, validation and error feedback.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Accepted input	CV upload: multipart PDF file, maximum 10 MB; Profile.id

Validation and Business Rules

- Only the owning freelancer can upload, view, replace or delete their CV.
- Uploads are PDF-only, at most 10 MB, and must have a valid PDF signature; reject missing, oversized or invalid files.
- Store CV URL, file name, public ID and upload time on FreelancerProfile. Do not expose private storage paths in public profile responses.
- A replacement updates metadata only after the new file is stored. A missing local file returns 404 without crashing the API.
- Deleting a CV clears its metadata and triggers a search-index update; it does not delete the profile or previously confirmed professional sections.

Normal Flow

1. The actor opens /freelancer/profile; /profiles/[id] and selects Upload CV.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Validate a PDF up to 10 MB and attach it to the owned FreelancerProfile.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Validate a PDF up to 10 MB and attach it to the owned FreelancerProfile.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful upload cv produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.15.2 Function: View CV (UC-07.04)

Function Trigger

The actor selects View CV. Navigation: /freelancer/profile; /profiles/[id].

Function Description

Actors / Roles

Freelancer.

Purpose

Authorize the owner and serve or redirect to the current CV; return 404 when its stored file is unavailable.

Interface and Data Processing

`GET /profiles/:id/cv/file`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Profile, FreelancerProfile, ProfileRevision. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

View CV

Current CV file name; Upload or Replace PDF picker; View/Download and Delete confirmation; progress, validation and error feedback.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Accepted input	CV upload: multipart PDF file, maximum 10 MB; Profile.id

Validation and Business Rules

- Only the owning freelancer can upload, view, replace or delete their CV.
- Uploads are PDF-only, at most 10 MB, and must have a valid PDF signature; reject missing, oversized or invalid files.
- Store CV URL, file name, public ID and upload time on FreelancerProfile. Do not expose private storage paths in public profile responses.
- A replacement updates metadata only after the new file is stored. A missing local file returns 404 without crashing the API.
- Deleting a CV clears its metadata and triggers a search-index update; it does not delete the profile or previously confirmed professional sections.

Normal Flow

1. The actor opens /freelancer/profile; /profiles/[id] and selects View CV.
2. The application submits the selected identifier or list filters.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Authorize the owner and serve or redirect to the current CV; return 404 when its stored file is unavailable.
5. The screen renders the response, or an explicit empty state for a valid empty collection.

Alternative Flow

Show an informative empty state for a valid empty result and allow retry after a transport failure.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
A1	A valid empty collection displays an empty state; it must not be treated as a missing account.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Authorize the owner and serve or redirect to the current CV; return 404 when its stored file is unavailable.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful view cv produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.15.3 Function: Replace CV (UC-07.05)

Function Trigger

The actor selects Replace CV. Navigation: /freelancer/profile; /profiles/[id].

Function Description

Actors / Roles

Freelancer.

Purpose

Store a valid replacement PDF, update CV metadata and clean up the previous file after a successful update.

Interface and Data Processing

`POST /profiles/:id/cv`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Profile, FreelancerProfile, ProfileRevision. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Replace CV

Current CV file name; Upload or Replace PDF picker; View/Download and Delete confirmation; progress, validation and error feedback.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Accepted input	CV upload: multipart PDF file, maximum 10 MB; Profile.id

Validation and Business Rules

- Only the owning freelancer can upload, view, replace or delete their CV.
- Uploads are PDF-only, at most 10 MB, and must have a valid PDF signature; reject missing, oversized or invalid files.
- Store CV URL, file name, public ID and upload time on FreelancerProfile. Do not expose private storage paths in public profile responses.
- A replacement updates metadata only after the new file is stored. A missing local file returns 404 without crashing the API.
- Deleting a CV clears its metadata and triggers a search-index update; it does not delete the profile or previously confirmed professional sections.

Normal Flow

1. The actor opens /freelancer/profile; /profiles/[id] and selects Replace CV.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Store a valid replacement PDF, update CV metadata and clean up the previous file after a successful update.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Store a valid replacement PDF, update CV metadata and clean up the previous file after a successful update.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful replace cv produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.15.4 Function: Delete CV (UC-07.06)

Function Trigger

The actor selects Delete CV. Navigation: /freelancer/profile; /profiles/[id].

Function Description

Actors / Roles

Freelancer.

Purpose

Clear CV URL, public ID, file name and upload time, and update the related search index.

Interface and Data Processing

`DELETE /profiles/:id/cv`. The browser calls the Next.js BFF, which forwards authenticated requests to the NestJS API. The server derives the acting user from authentication and accesses Profile, FreelancerProfile, ProfileRevision. Route IDs are validated independently from owner IDs.

Screen Layout (Mock-up Description)

Delete CV

Current CV file name; Upload or Replace PDF picker; View/Download and Delete confirmation; progress, validation and error feedback.

Schematic interface description, not an application screenshot.

Function Details

Input Data

Data	Definition
Accepted input	CV upload: multipart PDF file, maximum 10 MB; Profile.id

Validation and Business Rules

- Only the owning freelancer can upload, view, replace or delete their CV.
- Uploads are PDF-only, at most 10 MB, and must have a valid PDF signature; reject missing, oversized or invalid files.
- Store CV URL, file name, public ID and upload time on FreelancerProfile. Do not expose private storage paths in public profile responses.
- A replacement updates metadata only after the new file is stored. A missing local file returns 404 without crashing the API.
- Deleting a CV clears its metadata and triggers a search-index update; it does not delete the profile or previously confirmed professional sections.

Normal Flow

1. The actor opens /freelancer/profile; /profiles/[id] and selects Delete CV.
2. The application presents the current values or action confirmation; the actor supplies the required data and submits.
3. The API checks authentication where required, authorization, resource existence and input validity.
4. Clear CV URL, public ID, file name and upload time, and update the related search index.
5. After the transaction succeeds, the screen displays the result and reloads affected data; a pending revision is labeled as awaiting review.

Alternative Flow

Cancel before submission closes the form or confirmation without persisting changes. On failure, preserve user input for correction and retry.

Abnormal Flow / Exception Handling

Case	Description
E1	Missing or expired authentication, or insufficient role/ownership: deny the operation and do not change data.
E2	Missing, deleted or inaccessible resource: show a not-found result.
E3	Invalid input, duplicate relationship, or invalid workflow state: explain the problem and retain the entered values.
E4	Database, network or storage failure: report a retryable error; do not report success or terminate the API process.

Output Data

Data	Definition
Result	Clear CV URL, public ID, file name and upload time, and update the related search index.
Failure	Structured error without sensitive data; no successful UI state when the operation failed.

Acceptance Criteria

- Successful delete cv produces the documented response and survives a reload where data is persisted.
- An unauthorized actor cannot access or modify the resource.
- Invalid input and missing resources do not create a false success state or expose sensitive data.

2.16 Nhuy Requirement Traceability and Acceptance Scope

The following register covers the 51 use cases owned by Nguyen Lu Nhat Huy across Sprints 1–5. Use-case identifiers follow the project UC catalog. The specifications state required behavior; the acceptance-gap register distinguishes requirements from verified implementation. Screen layouts are schematic descriptions rather than fabricated screenshots.

Sprint	UC group	Feature	Persistent entities
1, 5	UC-07	Freelancer Profile Management	Profile, FreelancerProfile, ProfileRevision
1	UC-08	Freelancer Skills Management	Profile, FreelancerProfile, FreelancerSkill, Skill
1	UC-09	Portfolio Management	Profile, FreelancerProfile, PortfolioItem
2	UC-18	Identity Verification	User, IdVerificationDocument, FreelancerProfile
2	UC-19	Client Profile Management	User, Profile, ClientProfile, SocialLink, ProfileRevision
2	UC-20	Social Links Management	Profile, SocialLink
2	UC-21	Favorite Freelancers	User, Profile, FreelancerProfile, FavoriteFreelancer
3	UC-29	Following Management	User, FollowFreelancer, Notification
3	UC-30	Notification Management	Notification
3	UC-31	Role Switching and Enrollment	User, Role, UserRole, Session
4	UC-39	Contract Review Management	Contract, Review, Notification
4	UC-40	Review Response Management	Review, ReviewResponse, Notification
4	UC-41	General Account Profile	User, Profile, ProfileRevision
5	UC-47	Profile Revision Review and Expert Profile	ProfileRevision, Profile, FreelancerProfile, ClientProfile, Expert, Notification

2.16.1 Acceptance Gaps Requiring Verification

The following issues were identified in the earlier source/database audit. They are not asserted to be fixed by this documentation update. Revalidate them against the latest merged dev source.

UC	Acceptance gap
UC-18.01	Identity request uniqueness, confidential storage delivery and 5 MB/WebP validation remain acceptance gaps in the earlier audit. The AI UI accepts images and liveness; a manual PDF path needs separate UI verification.
UC-19.02; UC-20.02	HTTP/HTTPS-only URL validation and database-length limits require enforcement. Industry/location and portfolio media are excluded from the agreed database scope.
UC-20.01; UC-20.03	Public Freelancer social links and a confirmation step before deleting a social link require UI completion.
UC-21.01–02	Favorites must consistently display new freelancers after a successful add and exclude invalid targets.
All owned UCs	A complete SRS is not evidence of completed implementation. Repeat integration and browser acceptance tests on the latest dev revision before marking these requirements verified.

2.16.2 Schema and Identifier Conventions

- Profile.id identifies a general profile; User.id identifies an account. FreelancerProfile.id identifies the professional subtype. The three identifiers are not interchangeable.
- Client public profile routes and favorite/follow targets use User.id. Freelancer public profile routes use Profile.id.
- Contract URLs use the current slug route, while review APIs accept Contract.id.
- Portfolio scope is title, description, technologies and projectUrl. Client company scope is companyName, companyDescription and website.
- CV upload is PDF-only, maximum 10 MB. Avatar upload is JPEG/PNG/WebP, maximum 5 MB. Identity-document acceptance rules are recorded independently from these limits.